

TUI game collection - A3 - OOP

Generated by Doxygen 1.13.2

Chapter 1

Fun Box Collection

University C++ OOP Course. Terminal-based boardgame collection.

- SUS tic tac toe: generate as many "SUS" combinations as you can
- Four-in-a-Row: Connect four pieces in a row to win
- 5×5 Tic-Tac-Toe: Extended grid tactical gameplay
- Word Tic-Tac-Toe: Strategic word-building variant
- Misère Tic-Tac-Toe: Reverse objective: avoid three in a row
- Diamond Tic-Tac-Toe: Unique diamond-shaped board layout
- 4×4 Tic-Tac-Toe: Compact strategy challenge
- Pyramid Tic-Tac-Toe: Multi-level 3D gameplay
- Numerical Tic-Tac-Toe: Number-based strategic variant
- Obstacles Tic-Tac-Toe: Navigate blocked square
- Infinity Tic-Tac-Toe: Endless expandable grid
- Ultimate Tic-Tac-Toe: Meta-game with 9 sub-boards
- Memory Tic-Tac-Toe: Hidden pieces memory challenge

1.0.1 Extras

- Labyrinth: Navigate through maze challenges
- Spaceship: avoid the falling asteroids
- Snake: Classic arcade growth game

1.1 Quick Start

Prerequisites

C++17 compatible compiler (GCC 7+, Clang 5+, MSVC 2017+) CMake 3.10 or higher

Build Instructions bash# Clone the repository git clone <https://github.com/Nytril-ark/FunBox>.↵
git cd FunBox

1.1.1 Create build directory

mkdir build && cd build

1.1.2 Configure and build

cmake .. cmake --build .
or, on linux
cmake .. make

1.1.3 Run the application

./BoardGameApp

1.2 Screenshots

1.2.1 Menu

1.2.2 Obstacles Tic Tac Toe

1.2.3 Word Tic Tac Toe

1.2.4 Labyrinth

1.2.4.0.0.1 Developed by:

- . Mostafa Ahmed
- . Samir Kahlawy
- . Ammen
- . Abdallah Youssef

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Board< T >	??
Board< char >	??
DimaondTicTacToe_Board	??
FourByFour_Board	??
FourInARow_Board	??
Infinity_TicTacToe_Board	??
MemoryTicTacToeBoard	??
Mini_Board	??
MisereTicTacToeBoard	??
ObstaclesTicTacToeBoard	??
Pyramid_Board	??
SUS_Board	??
TicTacToe5x5_Board	??
Ultimate_Board	??
WordTicTacToe_board	??
X_O_Board	??
Board< int >	??
NumericalTicTacToe_Board	??
GameManager< T >	??
Menu	??
Move< T >	??
Player< T >	??
Player< char >	??
DimaondPlayer	??
Infinity_TicTacToe_Player	??
MisereAIPlayer	??
MiserePlayer	??
TicTacToe5x5_AIPlayer	??
WordTicTacToe_AIPlayer	??
Token	??
UI< T >	??
UI< char >	??
DimaondUI	??
FourByFour_UI	??
FourInARow_UI	??
Infinity_TicTacToe_UI	??
MemoryTicTacToeUI	??
Mini_UI	??
MisereTicTacToeUI	??

ObstaclesTicTacToeUI	??
Pyramid_UI	??
SUS_UI	??
TicTacToe5x5_UI	??
Ultimate_UI	??
WordTicTacToe_UI	??
XO_UI	??
UI< int >	??
NumericalTicTacToe_UI	??

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Board< T >	Base template for any board used in board games	??
DimaondPlayer		??
DimaondTicTacToe_Board		??
DimaondUI		??
FourByFour_Board		??
FourByFour_UI		??
FourInARow_Board		??
FourInARow_UI		??
GameManager< T >	Controls the flow of a board game between two players	??
Infinity_TicTacToe_Board	Represents the board for Infinity Tic Tac Toe	??
Infinity_TicTacToe_Player		??
Infinity_TicTacToe_UI	User interface for Infinity Tic Tac Toe	??
MemoryTicTacToeBoard	Implements a 3x3 Memory Tic Tac Toe board where symbols are hidden until the game ends	??
MemoryTicTacToeUI	Handles UI and user interactions for Memory Tic Tac Toe	??
Menu		??
Mini_Board		??
Mini_UI		??
MisereAIPlayer		??
MiserePlayer		??
MisereTicTacToeBoard	Represents the board for Misère Tic Tac Toe	??
MisereTicTacToeUI	User interface for Misère Tic Tac Toe	??
Move< T >	Represents a single move in a board game	??
NumericalTicTacToe_Board	Represents the board for the Numerical Tic Tac Toe game	??
NumericalTicTacToe_UI	Handles user interaction for the Numerical Tic Tac Toe game	??
ObstaclesTicTacToeBoard	Represents the board for Obstacles Tic Tac Toe	??
ObstaclesTicTacToeUI	User interface for Obstacles Tic Tac Toe	??

Player< T >		
	Base template for all players (human or AI)	??
Pyramid_Board		??
Pyramid_UI		??
SUS_Board		
	Represents the game board for the SUS game	??
SUS_UI		
	Handles user interaction for the SUS game	??
TicTacToe5x5_AIPlayer		??
TicTacToe5x5_Board		
	Represents a 5x5 TicTacToe board with scoring logic for sequences of 3	??
TicTacToe5x5_UI		
	Handles user interface for the 5x5 TicTacToe game	??
Token		??
UI< T >		
	Base class for handling user interface and input/output	??
Ultimate_Board		??
Ultimate_UI		??
WordTicTacToe_AIPlayer		??
WordTicTacToe_board		??
WordTicTacToe_UI		??
X_O_Board		
	Represents the Tic-Tac-Toe game board	??
XO_UI		
	User Interface class for the X-O (Tic-Tac-Toe) game	??

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

include/DiamondTicTacToe_Classes.h	??
include/FourInARow_Classes.h	??
include/InfinityTicTacToe_Classes.h	??
include/maze.h	??
include/MemoryTicTacToe_Classes.h	??
include/MisereTicTacToe_Classes.h	??
include/NumericalTicTacToe_Classes.h	??
include/ObstaclesTicTacToe_Classes.h	??
include/PyramidTicTacToe_Classes.h	??
include/snake.h	??
include/spaceship.h	??
include/SUS_Classes.h	??
include/TicTacToe4x4_Classes.h	??
include/TicTacToe5x5_Classes.h	??
include/UltimateTicTacToe_Classes.h	??
include/WordTicTacToe_Classes.h	??
include/XO_Classes.h	
Defines the X-O (Tic-Tac-Toe) specific classes that extend the generic board game framework	??
include/util/BoardGame_Classes.h	??
include/util/colors.h	??
include/util/GameLaunchers.h	??
include/util/Menu.h	??
include/util/util.h	??
src/abdallah/WordTicTacToe_Classes.cpp	
Implementation of Word Tic-Tac-Toe game classes	??
src/ammen/PyramidTicTacToe_Classes.cpp	
Implementation of Pyramid Tic-Tac-Toe game classes	??
src/ammen/UltimateTicTacToe_Classes.cpp	
Implementation of Ultimate Tic-Tac-Toe game classes	??

Chapter 5

Class Documentation

5.1 Board< T > Class Template Reference

Base template for any board used in board games.

```
#include <BoardGame_Classes.h>
```

Public Member Functions

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual **~Board** ()
Virtual destructor. Frees allocated board memory.
- virtual bool [update_board](#) ([Move](#)< T > *move)=0
Update the board with a new move.
- virtual bool [is_win](#) ([Player](#)< T > *)=0
Check if a player has won.
- virtual bool [is_lose](#) ([Player](#)< T > *)=0
Check if a player has lost.
- virtual bool [is_draw](#) ([Player](#)< T > *)=0
Check if the game ended in a draw.
- virtual bool [game_is_over](#) ([Player](#)< T > *)=0
Check if the game is over.
- vector< vector< T > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Protected Attributes

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< T > > **board**
2D vector for the board
- int **n_moves** = 0
Number of moves made.

5.1.1 Detailed Description

template<typename T>
class Board< T >

Base template for any board used in board games.

Template Parameters

T	Type of the elements stored on the board (e.g., char, int, string).
----------	---

Provides core data (rows, columns, matrix) and virtual methods to be implemented by specific games like Tic-Tac-Toe, Connect4, etc.

5.1.2 Member Function Documentation

5.1.2.1 game_is_over()

```
template<typename T>
virtual bool Board< T >::game_is_over (
    Player< T > * ) [pure virtual]
```

Check if the game is over.

Implemented in [DimaondTicTacToe_Board](#), [FourByFour_Board](#), [FourInARow_Board](#), [Infinity_TicTacToe_Board](#), [MemoryTicTacToeBoard](#), [Mini_Board](#), [MisereTicTacToeBoard](#), [NumericalTicTacToe_Board](#), [ObstaclesTicTacToeBoard](#), [Pyramid_Board](#), [SUS_Board](#), [TicTacToe5x5_Board](#), [Ultimate_Board](#), [WordTicTacToe_board](#), and [X_O_Board](#).

5.1.2.2 is_draw()

```
template<typename T>
virtual bool Board< T >::is_draw (
    Player< T > * ) [pure virtual]
```

Check if the game ended in a draw.

Implemented in [DimaondTicTacToe_Board](#), [FourByFour_Board](#), [FourInARow_Board](#), [Infinity_TicTacToe_Board](#), [MemoryTicTacToeBoard](#), [Mini_Board](#), [MisereTicTacToeBoard](#), [NumericalTicTacToe_Board](#), [ObstaclesTicTacToeBoard](#), [Pyramid_Board](#), [SUS_Board](#), [TicTacToe5x5_Board](#), [Ultimate_Board](#), [WordTicTacToe_board](#), and [X_O_Board](#).

5.1.2.3 is_lose()

```
template<typename T>
virtual bool Board< T >::is_lose (
    Player< T > * ) [pure virtual]
```

Check if a player has lost.

Implemented in [DimaondTicTacToe_Board](#), [FourByFour_Board](#), [FourInARow_Board](#), [Infinity_TicTacToe_Board](#), [MemoryTicTacToeBoard](#), [Mini_Board](#), [MisereTicTacToeBoard](#), [NumericalTicTacToe_Board](#), [ObstaclesTicTacToeBoard](#), [Pyramid_Board](#), [SUS_Board](#), [TicTacToe5x5_Board](#), [Ultimate_Board](#), [WordTicTacToe_board](#), and [X_O_Board](#).

5.1.2.4 is_win()

```
template<typename T>
virtual bool Board< T >::is_win (
    Player< T > * ) [pure virtual]
```

Check if a player has won.

Implemented in [DimaondTicTacToe_Board](#), [FourByFour_Board](#), [FourInARow_Board](#), [Infinity_TicTacToe_Board](#), [MemoryTicTacToeBoard](#), [Mini_Board](#), [MisereTicTacToeBoard](#), [NumericalTicTacToe_Board](#), [ObstaclesTicTacToeBoard](#), [Pyramid_Board](#), [SUS_Board](#), [TicTacToe5x5_Board](#), [Ultimate_Board](#), [WordTicTacToe_board](#), and [X_O_Board](#).

5.1.2.5 update_board()

```
template<typename T>
virtual bool Board< T >::update_board (
    Move< T > * move) [pure virtual]
```

Update the board with a new move.

Parameters

<code>move</code>	The move object containing position and symbol.
-------------------	---

Returns

true if the move is valid and applied, false otherwise.

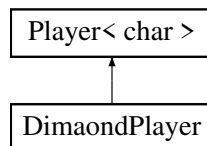
Implemented in [DimaondTicTacToe_Board](#), [FourByFour_Board](#), [FourInARow_Board](#), [Infinity_TicTacToe_Board](#), [MemoryTicTacToeBoard](#), [Mini_Board](#), [MisereTicTacToeBoard](#), [NumericalTicTacToe_Board](#), [ObstaclesTicTacToeBoard](#), [Pyramid_Board](#), [SUS_Board](#), [TicTacToe5x5_Board](#), [Ultimate_Board](#), [WordTicTacToe_board](#), and [X_O_Board](#).

The documentation for this class was generated from the following file:

- `include/util/BoardGame_Classes.h`

5.2 DimaondPlayer Class Reference

Inheritance diagram for DimaondPlayer:



Public Member Functions

- **DimaondPlayer** (string n, char s, PlayerType t)

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- [Board](#)< char > * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** ([Board](#)< char > *b)
Assign the board pointer for the player.

Additional Inherited Members

Protected Attributes inherited from [Player< char >](#)

- string **name**
[Player](#) name.
- PlayerType **type**
[Player](#) type (e.g., HUMAN or COMPUTER)
- char **symbol**

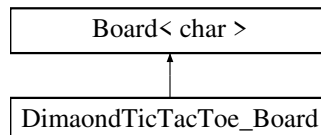
- *Players symbol on board.*
- `Board< char > * boardPtr`
Pointer to the game board.

The documentation for this class was generated from the following file:

- `include/DiamondTicTacToe_Classes.h`

5.3 DimaondTicTacToe_Board Class Reference

Inheritance diagram for DimaondTicTacToe_Board:



Public Member Functions

- `bool update_board (Move< char > *move)` override
Update the board with a player's move.
- `bool is_win (Player< char > *player)` override
Determine if a player has won the game. Winning requires 3 in a line plus 4 in another line.
- `bool is_lose (Player< char > *player)` override
Determine if a player has lost the game. Currently always returns false.
- `bool is_draw (Player< char > *player)` override
Check if the game is a draw.
- `bool game_is_over (Player< char > *player)` override
Check if the game is over (win or draw).
- `bool is_valid_cell (int x, int y)`
Verify if a cell is valid for a move.

Public Member Functions inherited from `Board< char >`

- `Board (int rows, int columns)`
Construct a board with given dimensions.
- `virtual ~Board ()`
Virtual destructor. Frees allocated board memory.
- `vector< vector< char > > get_board_matrix ()` const
Return a copy of the current board as a 2D vector.
- `int get_rows ()` const
Get number of rows.
- `int get_columns ()` const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from `Board< char >`

- `int rows`
Number of rows.
- `int columns`
Number of columns.

- `vector< vector< char > > board`
2D vector for the board
- `int n_moves`
Number of moves made.

5.3.1 Member Function Documentation

5.3.1.1 game_is_over()

```
bool DimaondTicTacToe_Board::game_is_over (
    Player< char > * player) [override], [virtual]
```

Check if the game is over (win or draw).

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the game is finished

Implements [Board< char >](#).

5.3.1.2 is_draw()

```
bool DimaondTicTacToe_Board::is_draw (
    Player< char > * player) [override], [virtual]
```

Check if the game is a draw.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if all cells are filled and no win condition is met

Implements [Board< char >](#).

5.3.1.3 is_lose()

```
bool DimaondTicTacToe_Board::is_lose (
    Player< char > * player) [override], [virtual]
```

Determine if a player has lost the game. Currently always returns false.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

false

Implements [Board< char >](#).

5.3.1.4 is_valid_cell()

```
bool DimaondTicTacToe_Board::is_valid_cell (
    int x,
    int y)
```

Verify if a cell is valid for a move.

Parameters

<i>x</i>	Row index
<i>y</i>	Column index

Returns

true if the cell is inside the board, inside the diamond, and empty

5.3.1.5 is_win()

```
bool DimaondTicTacToe_Board::is_win (
    Player< char > * player) [override], [virtual]
```

Determine if a player has won the game. Winning requires 3 in a line plus 4 in another line.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the player has won, false otherwise

Implements [Board< char >](#).

5.3.1.6 update_board()

```
bool DimaondTicTacToe_Board::update_board (
    Move< char > * move) [override], [virtual]
```

Update the board with a player's move.

Parameters

<i>move</i>	Pointer to the move object containing coordinates and symbol
-------------	--

Returns

true if the move was valid and applied, false otherwise

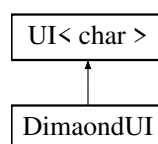
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/DiamondTicTacToe_Classes.h
- src/mostafa/DiamondTicTacToe_Classes.cpp

5.4 DimaondUI Class Reference

Inheritance diagram for DimaondUI:



Public Member Functions

- `Move< char > * get_move (Player< char > *player)` override
Get a move from a player (human or computer).
- `Player< char > * create_player (string &name, char symbol, PlayerType type)` override
Create a new player (human or computer).

Public Member Functions inherited from `UI< char >`

- `UI (string message, int cell_display_width)`
Construct the `UI` and display a welcome message.
- `void display_message (string message)`
Display any message to the user.
- `virtual Player< char > ** setup_players ()`
Set up players for the game.
- `void display_board_matrix (const vector< vector< char > > &matrix) const`
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from `UI< char >`

- `string get_player_name (string player_label)`
Ask the user for the player's name.
- `PlayerType get_player_type_choice (string player_label, const vector< string > &options)`
Ask the user to choose the player type from a list.

Protected Attributes inherited from `UI< char >`

- `int cell_width`
Width of each displayed board cell.

5.4.1 Member Function Documentation

5.4.1.1 `create_player()`

```
Player< char > * DimaondUI::create_player (
    string & name,
    char symbol,
    PlayerType type) [override], [virtual]
```

Create a new player (human or computer).

Parameters

<i>name</i>	<code>Player</code> name
<i>symbol</i>	<code>Player</code> symbol (X or O)
<i>type</i>	<code>Player</code> type (HUMAN or COMPUTER)

Returns

Pointer to the newly created player

Implements `UI< char >`.

5.4.1.2 get_move()

```
Move< char > * DimaondUI::get_move (  
    Player< char > * player)  [override], [virtual]
```

Get a move from a player (human or computer).

Parameters

<i>player</i>	Pointer to the player object
---------------	------------------------------

Returns

A new [Move](#) object containing coordinates and symbol

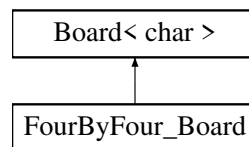
Implements [UI< char >](#).

The documentation for this class was generated from the following files:

- include/DiamondTicTacToe_Classes.h
- src/mostafa/DiamondTicTacToe_Classes.cpp

5.5 FourByFour_Board Class Reference

Inheritance diagram for FourByFour_Board:



Public Member Functions

- **FourByFour_Board** ()
Default constructor that initializes a 3x3 X-O board.
- bool **update_board** ([Move](#)< char > *move)
Updates the board with a player's move.
- bool **is_win** ([Player](#)< char > *player)
Checks if the given player has won the game.
- bool **is_lose** ([Player](#)< char > *)
Checks if the given player has lost the game.
- bool **is_draw** ([Player](#)< char > *player)
Checks if the game has ended in a draw.
- bool **game_is_over** ([Player](#)< char > *player)
Determines if the game is over (win or draw).
- void **putTokensAtPositions** ()
Updates token positions.
- void **display_winner** ([Player](#)< char > *player)
- void **display_draw** ()

Public Member Functions inherited from [Board< char >](#)

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual **~Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board< char >](#)

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.5.1 Member Function Documentation

5.5.1.1 game_is_over()

```
bool FourByFour_Board::game_is_over (
    Player< char > * player) [virtual]
```

Determines if the game is over (win or draw).

Parameters

<i>player</i>	Pointer to the player to evaluate.
---------------	------------------------------------

Returns

true if the game has ended, false otherwise.

Implements [Board< char >](#).

5.5.1.2 is_draw()

```
bool FourByFour_Board::is_draw (
    Player< char > * player) [virtual]
```

Checks if the game has ended in a draw.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

true if all cells are filled and no player has won, false otherwise.

Implements [Board< char >](#).

5.5.1.3 is_lose()

```
bool FourByFour_Board::is_lose (
    Player< char > * ) [inline], [virtual]
```

Checks if the given player has lost the game.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

Always returns false (not used in X-O logic).

Implements [Board< char >](#).

5.5.1.4 is_win()

```
bool FourByFour_Board::is_win (
    Player< char > * player) [virtual]
```

Checks if the given player has won the game.

Parameters

<code>player</code>	Pointer to the player being checked.
---------------------	--------------------------------------

Returns

true if the player has a winning line, false otherwise.

Implements [Board< char >](#).

5.5.1.5 update_board()

```
bool FourByFour_Board::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a player's move.

Parameters

<code>move</code>	Pointer to a Move<char> object containing move coordinates and symbol.
-------------------	--

Returns

true if the move is valid and successfully applied, false otherwise.

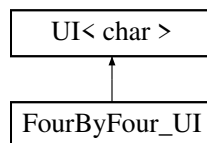
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/TicTacToe4x4_Classes.h
- src/ammen/TicTacToe4x4_Classes.cpp

5.6 FourByFour_UI Class Reference

Inheritance diagram for FourByFour_UI:



Public Member Functions

- [FourByFour_UI](#) ()
Constructs an [XO_UI](#) object.
- [~FourByFour_UI](#) ()
Destructor for [XO_UI](#).
- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Creates a player of the specified type.
- virtual [Move< char > * get_move](#) ([Player< char > *player](#))
Retrieves the next move from a player.
- [Player< char > ** setup_players](#) () override
Set up players for the game.

Public Member Functions inherited from [UI< char >](#)

- **UI** (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.6.1 Constructor & Destructor Documentation

5.6.1.1 FourByFour_UI()

`FourByFour_UI::FourByFour_UI ()`

Constructs an [XO_UI](#) object.

Initializes the base [UI<char>](#) class with the welcome message "FCAI X-O".

5.6.2 Member Function Documentation

5.6.2.1 create_player()

```
Player< char > * FourByFour_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a player of the specified type.

Parameters

<i>name</i>	Name of the player.
<i>symbol</i>	Character symbol ('X' or 'O') assigned to the player.
<i>type</i>	The type of the player (Human or Computer).

Returns

Pointer to the newly created [Player<char>](#) instance.

Implements [UI< char >](#).

5.6.2.2 get_move()

```
Move< char > * FourByFour_UI::get_move (
    Player< char > * player) [virtual]
```

Retrieves the next move from a player.

Parameters

<i>player</i>	Pointer to the player whose move is being requested.
---------------	--

Returns

A pointer to a new `Move<char>` object representing the player's action.

Implements `UI<char>`.

5.6.2.3 `setup_players()`

```
Player< char > ** FourByFour_UI::setup_players () [override], [virtual]
```

Set up players for the game.

Default implementation of setting up two players.

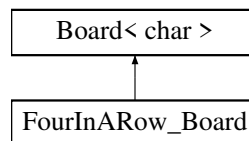
Reimplemented from `UI<char>`.

The documentation for this class was generated from the following files:

- include/TicTacToe4x4_Classes.h
- src/ammen/TicTacToe4x4_Classes.cpp

5.7 FourInARow_Board Class Reference

Inheritance diagram for FourInARow_Board:



Public Member Functions

- bool `update_board` (`Move<char> *move`)
Update the board with a new move.
- void `reset_board` ()
- int `calculate_score` (char sym)
- bool `is_win` (`Player<char> *player`)
Check if a player has won.
- bool `is_lose` (`Player<char> *`) override
Check if a player has lost.
- char `get_board_cell` (int row, int col) const
- bool `is_draw` (`Player<char> *player`)
Check if the game ended in a draw.
- bool `game_is_over` (`Player<char> *player`)
Check if the game is over.
- void `print_board` () const
- void `display_winner` (`Player<char> *player`) const
- void `display_draw` () const

Public Member Functions inherited from [Board< char >](#)

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual **~Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board< char >](#)

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.7.1 Member Function Documentation

5.7.1.1 game_is_over()

```
bool FourInARow_Board::game_is_over (
    Player< char > * ) [virtual]
```

Check if the game is over.

Implements [Board< char >](#).

5.7.1.2 is_draw()

```
bool FourInARow_Board::is_draw (
    Player< char > * ) [virtual]
```

Check if the game ended in a draw.

Implements [Board< char >](#).

5.7.1.3 is_lose()

```
bool FourInARow_Board::is_lose (
    Player< char > * ) [inline], [override], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.7.1.4 is_win()

```
bool FourInARow_Board::is_win (
    Player< char > * ) [virtual]
```

Check if a player has won.

Implements [Board< char >](#).

5.7.1.5 update_board()

```
bool FourInARow_Board::update_board (
    Move< char > * move) [virtual]
```

Update the board with a new move.

Parameters

<code>move</code>	The move object containing position and symbol.
-------------------	---

Returns

true if the move is valid and applied, false otherwise.

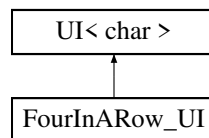
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/FourInARow_Classes.h
- src/samir/FourInARow_Classes.cpp

5.8 FourInARow_UI Class Reference

Inheritance diagram for FourInARow_UI:



Public Member Functions

- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Create a player object based on input name, symbol, and type.
- [Move< char > * get_move](#) ([Player< char > *player](#))
Ask the user (or AI) to make a move.
- [Player< char > ** setup_players](#) () override
Set up players for the game.

Public Member Functions inherited from [UI< char >](#)

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string [get_player_name](#) (string player_label)
Ask the user for the player's name.
- PlayerType [get_player_type_choice](#) (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- `int cell_width`
Width of each displayed board cell.

5.8.1 Member Function Documentation

5.8.1.1 `create_player()`

```
Player< char > * FourInARow_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Create a player object based on input name, symbol, and type.
Implements [UI< char >](#).

5.8.1.2 `get_move()`

```
Move< char > * FourInARow_UI::get_move (
    Player< char > * ) [virtual]
```

Ask the user (or AI) to make a move.
Implements [UI< char >](#).

5.8.1.3 `setup_players()`

```
Player< char > ** FourInARow_UI::setup_players () [override], [virtual]
```

Set up players for the game.
Default implementation of setting up two players.
Reimplemented from [UI< char >](#).

The documentation for this class was generated from the following files:

- `include/FourInARow_Classes.h`
- `src/samir/FourInARow_Classes.cpp`

5.9 `GameManager< T >` Class Template Reference

Controls the flow of a board game between two players.
`#include <BoardGame_Classes.h>`

Public Member Functions

- **`GameManager`** (`Board< T > *b`, `Player< T > *p[2]`, `UI< T > *u`)
Construct a game manager with board, players, and UI.
- `void run ()`
Run the main game loop until someone wins or the game ends.

5.9.1 Detailed Description

```
template<typename T>
class GameManager< T >
```

Controls the flow of a board game between two players.

Template Parameters

<i>T</i>	Type of symbol used on the board.
----------	-----------------------------------

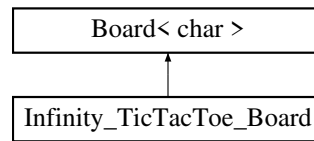
The documentation for this class was generated from the following file:

- `include/util/BoardGame_Classes.h`

5.10 Infinity_TicTacToe_Board Class Reference

Represents the board for Infinity Tic Tac Toe.

Inheritance diagram for Infinity_TicTacToe_Board:



Public Member Functions

- [Infinity_TicTacToe_Board](#) ()
- bool [update_board](#) ([Move](#)< char > *move) override
Update the board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player) override
Check if the player has won.
- bool [is_lose](#) ([Player](#)< char > *player) override
Check if the player has lost.
- bool [is_draw](#) ([Player](#)< char > *player) override
Check if the game is a draw.
- bool [game_is_over](#) ([Player](#)< char > *player) override
Check if the game is over.
- void [display_winner](#) ([Player](#)< char > *player)
- void [display_draw](#) ()

Public Member Functions inherited from [Board](#)< char >

- [Board](#) (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual [~Board](#) ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > [get_board_matrix](#) () const
Return a copy of the current board as a 2D vector.
- int [get_rows](#) () const
Get number of rows.
- int [get_columns](#) () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board](#)< char >

- int [rows](#)
Number of rows.
- int [columns](#)
Number of columns.
- vector< vector< char > > [board](#)
2D vector for the board
- int [n_moves](#)
Number of moves made.

5.10.1 Detailed Description

Represents the board for Infinity Tic Tac Toe.

This variant of Tic Tac Toe uses a 3x3 board but introduces special rules: after every 3 moves, the oldest move is removed.

5.10.2 Constructor & Destructor Documentation

5.10.2.1 Infinity_TicTacToe_Board()

```
Infinity_TicTacToe_Board::Infinity_TicTacToe_Board ()
< Initialize all cells as empty
```

5.10.3 Member Function Documentation

5.10.3.1 game_is_over()

```
bool Infinity_TicTacToe_Board::game_is_over (
    Player< char > * player) [override], [virtual]
```

Check if the game is over.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if win or draw

Implements [Board< char >](#).

5.10.3.2 is_draw()

```
bool Infinity_TicTacToe_Board::is_draw (
    Player< char > * player) [override], [virtual]
```

Check if the game is a draw.

A draw occurs when 13 moves have been played and no player has won.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if draw, false otherwise

Implements [Board< char >](#).

5.10.3.3 is_lose()

```
bool Infinity_TicTacToe_Board::is_lose (
    Player< char > * player) [override], [virtual]
```

Check if the player has lost.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

Always returns false (not implemented)

Implements [Board< char >](#).

5.10.3.4 is_win()

```
bool Infinity_TicTacToe_Board::is_win (
    Player< char > * player) [override], [virtual]
```

Check if the player has won.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the player has a winning line

Implements [Board< char >](#).

5.10.3.5 update_board()

```
bool Infinity_TicTacToe_Board::update_board (
    Move< char > * move) [override], [virtual]
```

Update the board with a player's move.

Handles normal moves, undo moves, and the special "infinity" rule where the oldest move is removed after every 3 moves.

Parameters

<i>move</i>	Pointer to the Move object containing coordinates and symbol
-------------	--

Returns

true if the move was valid and applied, false otherwise

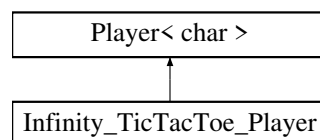
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/InfinityTicTacToe_Classes.h
- src/mostafa/InfinityTicTacToe_Classes.cpp

5.11 Infinity_TicTacToe_Player Class Reference

Inheritance diagram for Infinity_TicTacToe_Player:



Public Member Functions

- **Infinity_TicTacToe_Player** (string n, char s, PlayerType t)

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.

- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- Board< char > * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** (Board< char > *b)
Assign the board pointer for the player.

Additional Inherited Members

Protected Attributes inherited from Player< char >

- string **name**
Player name.
- PlayerType **type**
Player type (e.g., HUMAN or COMPUTER)
- char **symbol**
Players symbol on board.
- Board< char > * **boardPtr**
Pointer to the game board.

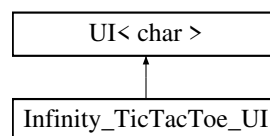
The documentation for this class was generated from the following file:

- include/InfinityTicTacToe_Classes.h

5.12 Infinity_TicTacToe_UI Class Reference

User interface for Infinity Tic Tac Toe.

Inheritance diagram for Infinity_TicTacToe_UI:



Public Member Functions

- Move< char > * **get_move** (Player< char > *player) override
Get a move from the player.
- Player< char > * **create_player** (string &name, char symbol, PlayerType type) override
Create a new player (human or computer).

Public Member Functions inherited from UI< char >

- UI (string message, int cell_display_width)
Construct the UI and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- virtual Player< char > ** **setup_players** ()
Set up players for the game.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.12.1 Detailed Description

User interface for Infinity Tic Tac Toe.

5.12.2 Member Function Documentation

5.12.2.1 create_player()

```
Player< char > * Infinity_TicTacToe_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [override], [virtual]
```

Create a new player (human or computer).

Parameters

<i>name</i>	Player name
<i>symbol</i>	Player symbol (X or O)
<i>type</i>	Player type

Returns

Pointer to the newly created player

Implements [UI< char >](#).

5.12.2.2 get_move()

```
Move< char > * Infinity_TicTacToe_UI::get_move (
    Player< char > * player) [override], [virtual]
```

Get a move from the player.

Human players enter coordinates manually. Computer players choose random coordinates.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

A new [Move](#) object

Implements [UI< char >](#).

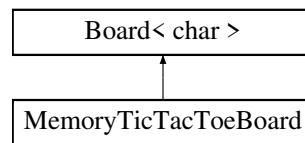
The documentation for this class was generated from the following files:

- include/InfinityTicTacToe_Classes.h
- src/mostafa/InfinityTicTacToe_Classes.cpp

5.13 MemoryTicTacToeBoard Class Reference

Implements a 3x3 Memory Tic Tac Toe board where symbols are hidden until the game ends.

Inheritance diagram for MemoryTicTacToeBoard:



Public Member Functions

- **MemoryTicTacToeBoard** ()
Constructs a 3x3 Memory Tic Tac Toe board and initializes both visible and memory boards.
- bool **update_board** (Move< char > *move)
Updates the board with a move, storing the real symbol in memory but showing a placeholder.
- bool **is_win** (Player< char > *player)
Checks if the given player has won by matching hidden (memory) symbols.
- bool **is_lose** (Player< char > *player)
Check if a player has lost.
- bool **is_draw** (Player< char > *player)
Checks if the game ended in a draw after all moves are played.
- bool **game_is_over** (Player< char > *player)
Determines if the game is over (win or draw).
- void **display_winner** (Player< char > *player)
Displays the winner and reveals the final memory board.
- void **display_draw** ()
Displays a draw message and reveals the final memory board.
- void **reveal_final_board** ()

Public Member Functions inherited from Board< char >

- **Board** (int rows, int columns)
Construct a board with given dimensions.
- virtual ~**Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from Board< char >

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.13.1 Detailed Description

Implements a 3x3 Memory Tic Tac Toe board where symbols are hidden until the game ends.

5.13.2 Member Function Documentation

5.13.2.1 display_winner()

```
void MemoryTicTacToeBoard::display_winner (
    Player< char > * player)
```

Displays the winner and reveals the final memory board.

Parameters

<i>player</i>	Pointer to the winning player.
---------------	--------------------------------

5.13.2.2 game_is_over()

```
bool MemoryTicTacToeBoard::game_is_over (
    Player< char > * player) [virtual]
```

Determines if the game is over (win or draw).

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if the game is over.

Implements [Board< char >](#).

5.13.2.3 is_draw()

```
bool MemoryTicTacToeBoard::is_draw (
    Player< char > * player) [virtual]
```

Checks if the game ended in a draw after all moves are played.

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if draw.

false otherwise.

Implements [Board< char >](#).

5.13.2.4 is_lose()

```
bool MemoryTicTacToeBoard::is_lose (
    Player< char > * ) [inline], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.13.2.5 is_win()

```
bool MemoryTicTacToeBoard::is_win (
    Player< char > * player) [virtual]
```

Checks if the given player has won by matching hidden (memory) symbols.

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if the player wins.
false otherwise.

Implements [Board< char >](#).

5.13.2.6 update_board()

```
bool MemoryTicTacToeBoard::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a move, storing the real symbol in memory but showing a placeholder.

Parameters

<i>move</i>	Pointer to a Move<char> containing coordinates and symbol.
-------------	--

Returns

true if the move was valid and successful.
false if the move was invalid or cell already taken.

Implements [Board< char >](#).

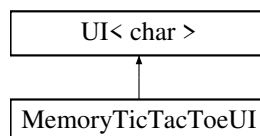
The documentation for this class was generated from the following files:

- include/MemoryTicTacToe_Classes.h
- src/group/MemoryTicTacToe_Classes.cpp

5.14 MemoryTicTacToeUI Class Reference

Handles [UI](#) and user interactions for Memory Tic Tac Toe.

Inheritance diagram for MemoryTicTacToeUI:

**Public Member Functions**

- **MemoryTicTacToeUI ()**
Constructs the [UI](#) with a welcome message.
- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Creates a player (human or computer).
- virtual [Move< char > * get_move](#) ([Player< char > *player](#))
Takes user input (row and column) and returns a [Move](#) object.

Public Member Functions inherited from UI< char >

- **UI** (string message, int cell_display_width)
Construct the UI and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- virtual **Player**< char > ** **setup_players** ()
Set up players for the game.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from UI< char >

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from UI< char >

- int **cell_width**
Width of each displayed board cell.

5.14.1 Detailed Description

Handles UI and user interactions for Memory Tic Tac Toe.

5.14.2 Member Function Documentation

5.14.2.1 create_player()

```
Player< char > * MemoryTicTacToeUI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a player (human or computer).

Parameters

<i>name</i>	Player name.
<i>symbol</i>	Player symbol (X or O).
<i>type</i>	Player type (HUMAN or COMPUTER).

Returns

Pointer to the created Player<char> object.

Implements UI< char >.

5.14.2.2 get_move()

```
Move< char > * MemoryTicTacToeUI::get_move (
    Player< char > * player) [virtual]
```

Takes user input (row and column) and returns a Move object.

Parameters

<code>player</code>	Pointer to the current player.
---------------------	--------------------------------

Returns

Pointer to a new `Move<char>` object.

Implements `UI<char>`.

The documentation for this class was generated from the following files:

- `include/MemoryTicTacToe_Classes.h`
- `src/group/MemoryTicTacToe_Classes.cpp`

5.15 Menu Class Reference

Public Member Functions

- void **addOption** (const std::string &option)
- int **display** (bool enhanced)
- bool **kbhit** ()
- int **displayMenu** (bool enhancedMenu)
- int **getOptionCount** () const

Static Public Attributes

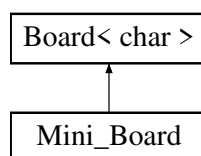
- static bool **noOptionChosen** = true

The documentation for this class was generated from the following files:

- `include/util/Menu.h`
- `src/Menu.cpp`

5.16 Mini_Board Class Reference

Inheritance diagram for `Mini_Board`:



Public Member Functions

- `Mini_Board` ()
MINI BOARD.
- bool `update_board` (`Move<char> *move`) override
Updates the mini-board with a player's move.
- bool `is_win` (`Player<char> *player`) override
Checks if the specified player has won this mini-board.
- bool `is_draw` (`Player<char> *player`) override
Checks if the mini-board has ended in a draw.
- bool `is_lose` (`Player<char> *player`) override
Checks if the specified player has lost this mini-board.
- bool `game_is_over` (`Player<char> *player`) override

- *Checks if the mini-board game has reached a terminal state.*
- char `get_winner` () const
Gets the winner of this mini-board.
- void `set_winner` (char w)
Sets the winner of this mini-board.
- bool `is_decided` () const
Checks if this mini-board has been decided.

Public Member Functions inherited from `Board< char >`

- `Board` (int rows, int columns)
Construct a board with given dimensions.
- virtual `~Board` ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > `get_board_matrix` () const
Return a copy of the current board as a 2D vector.
- int `get_rows` () const
Get number of rows.
- int `get_columns` () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from `Board< char >`

- int `rows`
Number of rows.
- int `columns`
Number of columns.
- vector< vector< char > > `board`
2D vector for the board
- int `n_moves`
Number of moves made.

5.16.1 Constructor & Destructor Documentation

5.16.1.1 Mini_Board()

```
Mini_Board::Mini_Board ()
```

MINI BOARD.

Default constructor for `Mini_Board`

Initializes a standard 3x3 Tic-Tac-Toe board with all cells set to blank_symbol. Each mini-board is part of the larger Ultimate Tic-Tac-Toe game.

Note

Calls the base `Board` constructor with dimensions 3x3

5.16.2 Member Function Documentation

5.16.2.1 game_is_over()

```
bool Mini_Board::game_is_over (
```

```
    Player< char > * player) [override], [virtual]
```

Checks if the mini-board game has reached a terminal state.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the board is won, drawn, or already decided
false if the game can continue

See also

[is_win\(\)](#)
[is_draw\(\)](#)
[is_decided\(\)](#)

Implements [Board< char >](#).

5.16.2.2 get_winner()

```
char Mini_Board::get_winner () const
```

Gets the winner of this mini-board.

Returns

char The winning player's symbol ('X', 'O'), 'D' for draw, or blank_symbol if undecided

5.16.2.3 is_decided()

```
bool Mini_Board::is_decided () const
```

Checks if this mini-board has been decided.

A mini-board is decided when it has been won by either player or ended in a draw.

Returns

true if the mini-board has a winner or is drawn
false if the mini-board is still in play

5.16.2.4 is_draw()

```
bool Mini_Board::is_draw (
    Player< char > * player) [override], [virtual]
```

Checks if the mini-board has ended in a draw.

A draw occurs when all 9 cells are filled and neither player has won. Creates temporary player objects to verify no winner exists.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the board is full with no winner
false if the game is not yet a draw or someone has won

===== // NOTE: test both those methods later to check

Implements [Board< char >](#).

5.16.2.5 is_lose()

```
bool Mini_Board::is_lose (
    Player< char > * player) [override], [virtual]
```

Checks if the specified player has lost this mini-board.

Parameters

<i>player</i>	Pointer to the Player object to check
---------------	---

Returns

false Always returns false (not used in current implementation)

Note

This method is not currently utilized in the game logic

Implements [Board< char >](#).

5.16.2.6 is_win()

```
bool Mini_Board::is_win (
    Player< char > * player) [override], [virtual]
```

Checks if the specified player has won this mini-board.

Examines all possible winning combinations: three horizontal rows, three vertical columns, and two diagonals.

Parameters

<i>player</i>	Pointer to the Player object to check for winning
---------------	---

Returns

true if the player has three symbols in a row

false if no winning combination exists

Implements [Board< char >](#).

5.16.2.7 set_winner()

```
void Mini_Board::set_winner (
    char w)
```

Sets the winner of this mini-board.

Parameters

<i>w</i>	The winner symbol to set ('X', 'O', or 'D' for draw)
----------	--

Note

Once set, the mini-board is considered decided and no further moves can be made

5.16.2.8 update_board()

```
bool Mini_Board::update_board (
    Move< char > * move) [override], [virtual]
```

Updates the mini-board with a player's move.

Validates and places a player's symbol on the mini-board at the specified position. Performs bounds checking, occupancy checking, and ensures the mini-board is not already decided (won or drawn).

Parameters

<i>move</i>	Pointer to Move object containing position (x,y) and player symbol
-------------	--

Returns

true if the move was valid and successfully placed
false if coordinates are invalid, cell is occupied, or board is already decided

Note

Increments the move counter on successful placement

See also

[is_decided\(\)](#)

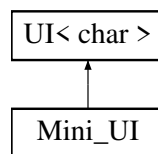
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/UltimateTicTacToe_Classes.h
- src/ammen/UltimateTicTacToe_Classes.cpp

5.17 Mini_UI Class Reference

Inheritance diagram for Mini_UI:

**Public Member Functions**

- [Player< char > *](#) [create_player](#) (string &name, char symbol, PlayerType type)
Create a player object based on input name, symbol, and type.
- virtual [Move< char > *](#) [get_move](#) ([Player< char > *](#)player)
Ask the user (or AI) to make a move.

Public Member Functions inherited from [UI< char >](#)

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- virtual [Player< char > **](#) [setup_players](#) ()
Set up players for the game.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members**Protected Member Functions inherited from [UI< char >](#)**

- string [get_player_name](#) (string player_label)
Ask the user for the player's name.
- PlayerType [get_player_type_choice](#) (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.17.1 Member Function Documentation

5.17.1.1 create_player()

```
Player< char > * Mini_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Create a player object based on input name, symbol, and type.

Implements [UI< char >](#).

5.17.1.2 get_move()

```
virtual Move< char > * Mini_UI::get_move (
    Player< char > * ) [virtual]
```

Ask the user (or AI) to make a move.

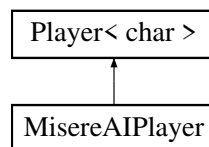
Implements [UI< char >](#).

The documentation for this class was generated from the following file:

- include/UltimateTicTacToe_Classes.h

5.18 MisereAIPlayer Class Reference

Inheritance diagram for MisereAIPlayer:



Public Member Functions

- **MisereAIPlayer** (string n, char s)
- pair< int, int > **get_best_move** ()

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- [Board](#)< char > * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** ([Board](#)< char > *b)
Assign the board pointer for the player.

Additional Inherited Members

Protected Attributes inherited from [Player< char >](#)

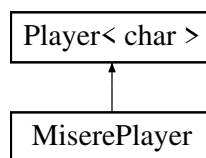
- string **name**
Player name.
- PlayerType **type**
Player type (e.g., HUMAN or COMPUTER)
- char **symbol**
Players symbol on board.
- [Board< char >](#) * **boardPtr**
Pointer to the game board.

The documentation for this class was generated from the following files:

- include/MisereTicTacToe_Classes.h
- src/mostafa/MisereTicTacToe_Classes.cpp

5.19 MiserePlayer Class Reference

Inheritance diagram for MiserePlayer:



Public Member Functions

- **MiserePlayer** (string n, char s, PlayerType t)

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- [Board< char >](#) * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** ([Board< char >](#) *b)
Assign the board pointer for the player.

Additional Inherited Members

Protected Attributes inherited from [Player< char >](#)

- string **name**
Player name.
- PlayerType **type**
Player type (e.g., HUMAN or COMPUTER)

- char **symbol**
Players symbol on board.
- [Board](#)< char > * **boardPtr**
Pointer to the game board.

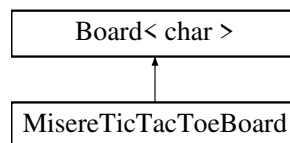
The documentation for this class was generated from the following file:

- include/MisereTicTacToe_Classes.h

5.20 MisereTicTacToeBoard Class Reference

Represents the board for Misère Tic Tac Toe.

Inheritance diagram for MisereTicTacToeBoard:



Public Member Functions

- [MisereTicTacToeBoard](#) ()
- bool [update_board](#) ([Move](#)< char > *move) override
Update the board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player) override
Check if the player has won.
- bool [is_lose](#) ([Player](#)< char > *player) override
Check if the player has lost.
- bool [is_draw](#) ([Player](#)< char > *player) override
Check if the game is a draw.
- bool [game_is_over](#) ([Player](#)< char > *player) override
Check if the game is over.

Public Member Functions inherited from [Board](#)< char >

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual ~**Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board](#)< char >

- int **rows**
Number of rows.
- int **columns**

- *Number of columns.*
vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.20.1 Detailed Description

Represents the board for Misère Tic Tac Toe.

Misère Tic Tac Toe is a variant where the goal is NOT to win. A player loses if they complete three in a row.

5.20.2 Constructor & Destructor Documentation

5.20.2.1 MisereTicTacToeBoard()

```
MisereTicTacToeBoard::MisereTicTacToeBoard ()
< Initialize all cells as empty
```

5.20.3 Member Function Documentation

5.20.3.1 game_is_over()

```
bool MisereTicTacToeBoard::game_is_over (
    Player< char > * player) [override], [virtual]
```

Check if the game is over.

The game ends when a player loses or when the game is a draw.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the game is finished

Implements [Board< char >](#).

5.20.3.2 is_draw()

```
bool MisereTicTacToeBoard::is_draw (
    Player< char > * player) [override], [virtual]
```

Check if the game is a draw.

A draw occurs when all cells are filled and no player has three in a row.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if draw, false otherwise

Implements [Board< char >](#).

5.20.3.3 is_lose()

```
bool MisereTicTacToeBoard::is_lose (
    Player< char > * player) [override], [virtual]
```

Check if the player has lost.

A player loses if they complete three in a row.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the player has three in a row, false otherwise

Implements [Board< char >](#).

5.20.3.4 is_win()

```
bool MisereTicTacToeBoard::is_win (
    Player< char > * player) [override], [virtual]
```

Check if the player has won.

In Misère Tic Tac Toe, winning is not the goal. This function always returns false.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

false

Implements [Board< char >](#).

5.20.3.5 update_board()

```
bool MisereTicTacToeBoard::update_board (
    Move< char > * move) [override], [virtual]
```

Update the board with a player's move.

Handles normal moves and undo moves.

Parameters

<i>move</i>	Pointer to the Move object containing coordinates and symbol
-------------	--

Returns

true if the move was valid and applied, false otherwise

< Undo move

< Apply move

Implements [Board< char >](#).

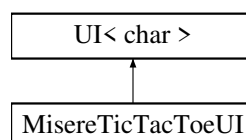
The documentation for this class was generated from the following files:

- include/MisereTicTacToe_Classes.h
- src/mostafa/MisereTicTacToe_Classes.cpp

5.21 MisereTicTacToeUI Class Reference

User interface for Misère Tic Tac Toe.

Inheritance diagram for MisereTicTacToeUI:



Public Member Functions

- `Move< char > * get_move (Player< char > *player)` override
Get a move from the player.
- `Player< char > * create_player (string &name, char symbol, PlayerType type)` override
Create a new player (human or computer).
- `Player< char > ** setup_players ()` override
Set up players for the game.

Public Member Functions inherited from `UI< char >`

- `UI (string message, int cell_display_width)`
Construct the `UI` and display a welcome message.
- `void display_message (string message)`
Display any message to the user.
- `void display_board_matrix (const vector< vector< char > > &matrix) const`
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from `UI< char >`

- `string get_player_name (string player_label)`
Ask the user for the player's name.
- `PlayerType get_player_type_choice (string player_label, const vector< string > &options)`
Ask the user to choose the player type from a list.

Protected Attributes inherited from `UI< char >`

- `int cell_width`
Width of each displayed board cell.

5.21.1 Detailed Description

User interface for Misère Tic Tac Toe.

5.21.2 Member Function Documentation

5.21.2.1 `create_player()`

```
Player< char > * MisereTicTacToeUI::create_player (
    string & name,
    char symbol,
    PlayerType type) [override], [virtual]
```

Create a new player (human or computer).

Parameters

<i>name</i>	<code>Player</code> name
<i>symbol</i>	<code>Player</code> symbol (X or O)
<i>type</i>	<code>Player</code> type

Returns

Pointer to the newly created player

Implements `UI< char >`.

5.21.2.2 get_move()

```
Move< char > * MisereTicTacToeUI::get_move (
    Player< char > * player) [override], [virtual]
```

Get a move from the player.

Human players enter coordinates manually. Computer players choose random coordinates.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

A new [Move](#) object

Implements [UI< char >](#).

5.21.2.3 setup_players()

```
Player< char > ** MisereTicTacToeUI::setup_players () [override], [virtual]
```

Set up players for the game.

Default implementation of setting up two players.

Reimplemented from [UI< char >](#).

The documentation for this class was generated from the following files:

- include/MisereTicTacToe_Classes.h
- src/mostafa/MisereTicTacToe_Classes.cpp

5.22 Move< T > Class Template Reference

Represents a single move in a board game.

```
#include <BoardGame_Classes.h>
```

Public Member Functions

- **Move** (int x, int y, T symbol)
Construct a move at (x, y) using a symbol.
- int **get_x** () const
Get row index.
- int **get_y** () const
Get column index.
- T **get_symbol** () const
Get the move symbol.

5.22.1 Detailed Description

```
template<typename T>
```

```
class Move< T >
```

Represents a single move in a board game.

Template Parameters

<i>T</i>	Type of symbol placed on the board (e.g., char, int).
----------	---

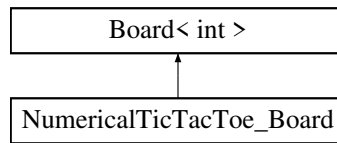
The documentation for this class was generated from the following file:

- include/util/BoardGame_Classes.h

5.23 NumericalTicTacToe_Board Class Reference

Represents the board for the Numerical Tic Tac Toe game.

Inheritance diagram for NumericalTicTacToe_Board:



Public Member Functions

- bool **update_board** (Move< int > *move) override
Updates the board with a given move.
- bool **is_win** (Player< int > *player) override
Determines if the given player has won.
- bool **is_lose** (Player< int > *player) override
Check if a player has lost.
- bool **is_draw** (Player< int > *player) override
Determines if the game is a draw.
- bool **game_is_over** (Player< int > *player) override
Checks if the game is over.
- int **calculate_score** (int number)
Calculates score for a given number.
- bool **is_valid_number** (Player< int > *player, int number)
Checks if a number is valid for the given player.
- bool **check_sum_15** ()
Checks if any row, column, or diagonal sums to 15.
- int **get_cell** (int r, int c) const
Gets the value of a cell.
- bool **is_used** (int n) const
Checks if a number has already been used.
- void **display_winner** (Player< int > *player)
- void **display_draw** ()

Public Member Functions inherited from Board< int >

- **Board** (int rows, int columns)
Construct a board with given dimensions.
- virtual ~**Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< int > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board< int >](#)

- `int rows`
Number of rows.
- `int columns`
Number of columns.
- `vector< vector< int > > board`
2D vector for the board
- `int n_moves`
Number of moves made.

5.23.1 Detailed Description

Represents the board for the Numerical Tic Tac Toe game.

The board is a 3x3 grid where players place numbers (1–9). [Player](#) 1 uses odd numbers, [Player](#) 2 uses even numbers. A player wins if any row, column, or diagonal sums to 15.

5.23.2 Member Function Documentation

5.23.2.1 `calculate_score()`

```
int NumericalTicTacToe_Board::calculate_score (
    int number)
```

Calculates score for a given number.

In this implementation, the score is simply the number itself.

Parameters

<i>number</i>	The number played.
---------------	--------------------

Returns

The score value.

5.23.2.2 `check_sum_15()`

```
bool NumericalTicTacToe_Board::check_sum_15 ()
```

Checks if any row, column, or diagonal sums to 15.

Returns

true if a winning condition is met, false otherwise.

5.23.2.3 `game_is_over()`

```
bool NumericalTicTacToe_Board::game_is_over (
    Player< int > * player) [override], [virtual]
```

Checks if the game is over.

The game ends if there is a win or a draw.

Parameters

<i>player</i>	Pointer to the Player<int> object.
---------------	--

Returns

true if the game is over, false otherwise.

Implements [Board< int >](#).

5.23.2.4 get_cell()

```
int NumericalTicTacToe_Board::get_cell (
    int r,
    int c) const
```

Gets the value of a cell.

Parameters

<i>r</i>	Row index.
<i>c</i>	Column index.

Returns

The number in the cell, or 0 if empty.

5.23.2.5 is_draw()

```
bool NumericalTicTacToe_Board::is_draw (
    Player< int > * player) [override], [virtual]
```

Determines if the game is a draw.

A draw occurs when the board is full and no winning condition is met.

Parameters

<i>player</i>	Pointer to the Player<int> object.
---------------	--

Returns

true if the game is a draw, false otherwise.

Implements [Board< int >](#).

5.23.2.6 is_lose()

```
bool NumericalTicTacToe_Board::is_lose (
    Player< int > * ) [inline], [override], [virtual]
```

Check if a player has lost.

Implements [Board< int >](#).

5.23.2.7 is_used()

```
bool NumericalTicTacToe_Board::is_used (
    int n) const
```

Checks if a number has already been used.

Parameters

<i>n</i>	The number to check.
----------	----------------------

Returns

true if the number is used, false otherwise.

5.23.2.8 is_valid_number()

```
bool NumericalTicTacToe_Board::is_valid_number (
    Player< int > * player,
    int number)
```

Checks if a number is valid for the given player.

- Must be between 1 and 9.
- Must not have been used before.
- [Player](#) 1 can only use odd numbers.
- [Player](#) 2 can only use even numbers.

Parameters

<i>player</i>	Pointer to the player making the move.
<i>number</i>	The number to validate.

Returns

true if the number is valid, false otherwise.

5.23.2.9 is_win()

```
bool NumericalTicTacToe_Board::is_win (
    Player< int > * player) [override], [virtual]
```

Determines if the given player has won.

Parameters

<i>player</i>	Pointer to the Player<int> object.
---------------	--

Returns

true if the player has won, false otherwise.

Implements [Board< int >](#).

5.23.2.10 update_board()

```
bool NumericalTicTacToe_Board::update_board (
    Move< int > * move) [override], [virtual]
```

Updates the board with a given move.

Validates the move before applying it. Marks the number as used.

Parameters

<i>move</i>	Pointer to the Move<int> object.
-------------	--

Returns

true if the move was successfully applied, false otherwise.

Implements [Board< int >](#).

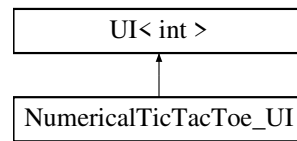
The documentation for this class was generated from the following files:

- include/NumericalTicTacToe_Classes.h
- src/samir/NumericalTicTacToe_Classes.cpp

5.24 NumericalTicTacToe_UI Class Reference

Handles user interaction for the Numerical Tic Tac Toe game.

Inheritance diagram for NumericalTicTacToe_UI:



Public Member Functions

- **NumericalTicTacToe_UI** (**NumericalTicTacToe_Board** *b)
- **Player**< int > * **create_player** (string &name, int symbol, PlayerType type) override
Creates a player object.
- **Player**< int > ** **setup_players** () override
Sets up the two players for the game.
- **Move**< int > * **get_move** (**Player**< int > *player) override
Gets the next move from a player.

Public Member Functions inherited from **UI**< int >

- **UI** (string message, int cell_display_width)
*Construct the **UI** and display a welcome message.*
- void **display_message** (string message)
Display any message to the user.
- void **display_board_matrix** (const vector< vector< int > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from **UI**< int >

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from **UI**< int >

- int **cell_width**
Width of each displayed board cell.

5.24.1 Detailed Description

Handles user interaction for the Numerical Tic Tac Toe game.

Provides methods to create players and get moves from them.

5.24.2 Member Function Documentation

5.24.2.1 create_player()

```

Player< int > * NumericalTicTacToe_UI::create_player (
    string & name,

```

```
int symbol,
PlayerType type) [override], [virtual]
```

Creates a player object.

Parameters

<i>name</i>	The name of the player.
<i>symbol</i>	The symbol (1 for odd, 2 for even).
<i>type</i>	The type of player (Human or Computer).

Returns

Pointer to the created [Player<int>](#) object.

Implements [UI< int >](#).

5.24.2.2 get_move()

```
Move< int > * NumericalTicTacToe_UI::get_move (
    Player< int > * player) [override], [virtual]
```

Gets the next move from a player.

- If the player is computer: chooses the first available cell and valid number.
- If the player is human: prompts for input (row, column, number).

Parameters

<i>player</i>	Pointer to the Player<int> object.
---------------	--

Returns

Pointer to the created [Move<int>](#) object.

Implements [UI< int >](#).

5.24.2.3 setup_players()

```
Player< int > ** NumericalTicTacToe_UI::setup_players () [override], [virtual]
```

Sets up the two players for the game.

[Player](#) 1 uses odd numbers, [Player](#) 2 uses even numbers.

Returns

Pointer array of two [Player<int>](#) objects.

Reimplemented from [UI< int >](#).

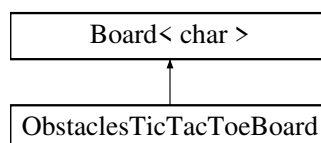
The documentation for this class was generated from the following files:

- include/NumericalTicTacToe_Classes.h
- src/samir/NumericalTicTacToe_Classes.cpp

5.25 ObstaclesTicTacToeBoard Class Reference

Represents the board for Obstacles Tic Tac Toe.

Inheritance diagram for ObstaclesTicTacToeBoard:



Public Member Functions

- [ObstaclesTicTacToeBoard](#) ()
- bool [update_board](#) ([Move](#)< char > *move)
Update the board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player)
Check if the player has won.
- bool [is_lose](#) ([Player](#)< char > *player)
Check if a player has lost.
- bool [is_draw](#) ([Player](#)< char > *player)
Check if the game is a draw.
- bool [game_is_over](#) ([Player](#)< char > *player)
Check if the game is over.
- void [display_winner](#) ([Player](#)< char > *player)
Display the winner message.
- void [display_draw](#) ()
Display the draw message.

Public Member Functions inherited from [Board](#)< char >

- [Board](#) (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual [~Board](#) ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > [get_board_matrix](#) () const
Return a copy of the current board as a 2D vector.
- int [get_rows](#) () const
Get number of rows.
- int [get_columns](#) () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from [Board](#)< char >

- int [rows](#)
Number of rows.
- int [columns](#)
Number of columns.
- vector< vector< char > > [board](#)
2D vector for the board
- int [n_moves](#)
Number of moves made.

5.25.1 Detailed Description

Represents the board for Obstacles Tic Tac Toe.

This variant uses a 6x6 board. When a player places 'O', two random empty cells are converted into obstacles.

5.25.2 Constructor & Destructor Documentation

5.25.2.1 [ObstaclesTicTacToeBoard](#)()

```
ObstaclesTicTacToeBoard::ObstaclesTicTacToeBoard ()  
< Initialize all cells as blank
```

5.25.3 Member Function Documentation

5.25.3.1 display_winner()

```
void ObstaclesTicTacToeBoard::display_winner (
    Player< char > * player)
```

Display the winner message.

Parameters

<i>player</i>	Pointer to the winning player
---------------	-------------------------------

5.25.3.2 game_is_over()

```
bool ObstaclesTicTacToeBoard::game_is_over (
    Player< char > * player) [virtual]
```

Check if the game is over.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if win or draw

Implements [Board< char >](#).

5.25.3.3 is_draw()

```
bool ObstaclesTicTacToeBoard::is_draw (
    Player< char > * player) [virtual]
```

Check if the game is a draw.

A draw occurs when all cells are filled and no player has won.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if draw, false otherwise

Implements [Board< char >](#).

5.25.3.4 is_lose()

```
bool ObstaclesTicTacToeBoard::is_lose (
    Player< char > * ) [inline], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.25.3.5 is_win()

```
bool ObstaclesTicTacToeBoard::is_win (
    Player< char > * player) [virtual]
```

Check if the player has won.

A player wins if they have four consecutive symbols in a row, column, or diagonal.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

true if the player has won, false otherwise

Implements [Board< char >](#).

5.25.3.6 update_board()

```
bool ObstaclesTicTacToeBoard::update_board (
    Move< char > * move) [virtual]
```

Update the board with a player's move.

If the move is valid, applies the symbol. If the symbol is 'O', two random empty cells are turned into obstacles.

Parameters

<i>move</i>	Pointer to the Move object containing coordinates and symbol
-------------	--

Returns

true if the move was valid and applied, false otherwise

Implements [Board< char >](#).

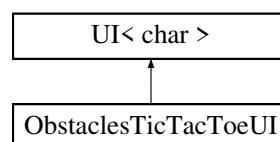
The documentation for this class was generated from the following files:

- include/ObstaclesTicTacToe_Classes.h
- src/abdallah/ObstaclesTicTacToe_Classes.cpp

5.26 ObstaclesTicTacToeUI Class Reference

User interface for Obstacles Tic Tac Toe.

Inheritance diagram for ObstaclesTicTacToeUI:



Public Member Functions

- [Player< char >](#) * [create_player](#) (string &name, char symbol, PlayerType type)
Create a new player (human or computer).
- virtual [Move< char >](#) * [get_move](#) ([Player< char >](#) *player)
Get a move from the player.

Public Member Functions inherited from [UI< char >](#)

- [UI](#) (string message, int cell_display_width)
Construct the UI and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- virtual [Player< char >](#) ** [setup_players](#) ()
Set up players for the game.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.26.1 Detailed Description

User interface for Obstacles Tic Tac Toe.

5.26.2 Member Function Documentation

5.26.2.1 create_player()

```
Player< char > * ObstaclesTicTacToeUI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Create a new player (human or computer).

Parameters

<i>name</i>	Player name
<i>symbol</i>	Player symbol (X or O)
<i>type</i>	Player type

Returns

Pointer to the newly created player

Implements [UI< char >](#).

5.26.2.2 get_move()

```
Move< char > * ObstaclesTicTacToeUI::get_move (
    Player< char > * player) [virtual]
```

Get a move from the player.

Human players enter coordinates manually. Computer players choose random coordinates.

Parameters

<i>player</i>	Pointer to the player
---------------	-----------------------

Returns

A new [Move](#) object

Implements [UI< char >](#).

The documentation for this class was generated from the following files:

- include/ObstaclesTicTacToe_Classes.h
- src/abdallah/ObstaclesTicTacToe_Classes.cpp

5.27 Player< T > Class Template Reference

Base template for all players (human or AI).

```
#include <BoardGame_Classes.h>
```

Public Member Functions

- **Player** (string n, T s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- T **get_symbol** () const
Get the player's symbol.
- Board< T > * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** (Board< T > *b)
Assign the board pointer for the player.

Protected Attributes

- string **name**
Player name.
- PlayerType **type**
Player type (e.g., HUMAN or COMPUTER)
- T **symbol**
Players symbol on board.
- Board< T > * **boardPtr**
Pointer to the game board.

5.27.1 Detailed Description

```
template<typename T>
```

```
class Player< T >
```

Base template for all players (human or AI).

Template Parameters

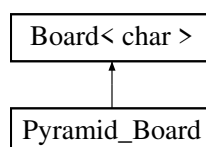
<i>T</i>	Type of symbol used by the player.
----------	------------------------------------

The documentation for this class was generated from the following file:

- include/util/BoardGame_Classes.h

5.28 Pyramid_Board Class Reference

Inheritance diagram for Pyramid_Board:



Public Member Functions

- [Pyramid_Board](#) ()
Default constructor that initializes a 3x3 X-O board.
- bool [update_board](#) ([Move](#)< char > *move)
Updates the board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player)
Checks if the given player has won the game.
- bool [is_lose](#) ([Player](#)< char > *)
Checks if the given player has lost the game.
- bool [is_draw](#) ([Player](#)< char > *player)
Checks if the game has ended in a draw.
- bool [game_is_over](#) ([Player](#)< char > *player)
Determines if the game is over (win or draw).
- void [display_winner](#) ([Player](#)< char > *player)
- void [display_draw](#) ()

Public Member Functions inherited from [Board](#)< char >

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual ~**Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Friends

- bool [isPrintable](#) (int y, int x)
Checks if a position is within the pyramid's playable area.
- void [initPrintableLocations](#) ()
Initializes the list of valid playable positions on the pyramid board.

Additional Inherited Members

Protected Attributes inherited from [Board](#)< char >

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.28.1 Constructor & Destructor Documentation

5.28.1.1 Pyramid_Board()

```
Pyramid_Board::Pyramid_Board ()
```

Default constructor that initializes a 3x3 X-O board.

Default constructor for [Pyramid_Board](#).

Initializes a 5x5 board with pyramid-shaped playable positions. The board has 9 valid positions arranged in a pyramid:

- Top level (row 2): 1 position
- Middle level (row 3): 3 positions
- Bottom level (row 4): 5 positions

All non-playable positions are marked with '*', and playable positions are initialized with the blank symbol.

Note

Calls the base [Board](#) constructor with dimensions 5x5

See also

[initPrintableLocations\(\)](#)

5.28.2 Member Function Documentation

5.28.2.1 game_is_over()

```
bool Pyramid_Board::game_is_over (
    Player< char > * player) [virtual]
```

Determines if the game is over (win or draw).

Checks if the game has reached a terminal state.

Parameters

<i>player</i>	Pointer to the player to evaluate.
---------------	------------------------------------

Returns

true if the game has ended, false otherwise.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the game is won or drawn

false if the game can continue

See also

[is_win\(\)](#)

[is_draw\(\)](#)

Implements [Board< char >](#).

5.28.2.2 is_draw()

```
bool Pyramid_Board::is_draw (
    Player< char > * player) [virtual]
```

Checks if the game has ended in a draw.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

true if all cells are filled and no player has won, false otherwise.

A draw occurs when all 9 playable positions are filled and no player has won.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the game is a draw (9 moves made and no winner)

false if the game is not yet a draw

Implements [Board< char >](#).

5.28.2.3 is_lose()

```
bool Pyramid_Board::is_lose (
    Player< char > * ) [inline], [virtual]
```

Checks if the given player has lost the game.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

Always returns false (not used in X-O logic).

Implements [Board< char >](#).

5.28.2.4 is_win()

```
bool Pyramid_Board::is_win (
    Player< char > * player) [virtual]
```

Checks if the given player has won the game.

Checks if the specified player has won the game.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

true if the player has a winning line, false otherwise.

Examines all possible winning combinations on the pyramid board:

- Three horizontal lines on the bottom row
- One horizontal line on the middle row
- One vertical line through the center
- Two diagonal lines from bottom corners to top
- Two diagonal lines from top to bottom corners

Parameters

<i>player</i>	Pointer to the Player object to check for winning
---------------	---

Returns

true if the player has three symbols in a row
false if no winning combination exists

Note

Total of 9 possible winning combinations are checked

Implements [Board< char >](#).

5.28.2.5 update_board()

```
bool Pyramid_Board::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a player's move.

Parameters

<i>move</i>	Pointer to a Move<char> object containing move coordinates and symbol.
-------------	--

Returns

true if the move is valid and successfully applied, false otherwise.

Validates and places a player's symbol on the board at the specified position. The move is only valid if the target cell is currently blank.

Parameters

<i>move</i>	Pointer to Move object containing position (x,y) and player symbol
-------------	--

Returns

true if the move was valid and successfully placed
false if the cell is already occupied

Note

Automatically converts the symbol to uppercase
Increments the move counter on successful placement

Implements [Board< char >](#).

5.28.3 Friends And Related Symbol Documentation**5.28.3.1 initPrintableLocations**

```
void initPrintableLocations () [friend]
```

Initializes the list of valid playable positions on the pyramid board.

Populates the Pyramid_Board::printable vector with all 9 valid positions that form the pyramid shape:

- Row 2: position (2,2) - pyramid top
- Row 3: positions (3,1), (3,2), (3,3) - pyramid middle
- Row 4: positions (4,0), (4,1), (4,2), (4,3), (4,4) - pyramid base

Note

This function should be called once during board initialization
 Modifies the static Pyramid_Board::printable vector

5.28.3.2 isPrintable

```
bool isPrintable (
    int y,
    int x) [friend]
```

Checks if a position is within the pyramid's playable area.
 Validates whether the given coordinates correspond to a valid position in the pyramid structure by checking against the list of printable positions.

Parameters

y	Row coordinate of the position to check
x	Column coordinate of the position to check

Returns

true if the position is playable (within pyramid boundaries)
 false if the position is outside the pyramid or non-playable

Note

Uses the static Pyramid_Board::printable vector for validation

See also

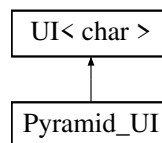
[initPrintableLocations\(\)](#)

The documentation for this class was generated from the following files:

- include/PyramidTicTacToe_Classes.h
- src/ammen/PyramidTicTacToe_Classes.cpp

5.29 Pyramid_UI Class Reference

Inheritance diagram for Pyramid_UI:

**Public Member Functions**

- [Pyramid_UI \(\)](#)
Constructs an [XO_UI](#) object.
- [~Pyramid_UI \(\)](#)
Destructor for [XO_UI](#).
- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Creates a player of the specified type.
- virtual [Move< char > * get_move](#) ([Player< char > *player](#)) override
Retrieves the next move from a player.
- [Player< char > ** setup_players \(\)](#) override
Sets up both players for the game.

Public Member Functions inherited from [UI< char >](#)

- **UI** (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.29.1 Constructor & Destructor Documentation

5.29.1.1 Pyramid_UI()

`Pyramid_UI::Pyramid_UI ()`

Constructs an [XO_UI](#) object.

Default constructor for [Pyramid_UI](#).

Initializes the base [UI<char>](#) class with the welcome message "FCAI X-O".

Initializes the user interface with a welcome message and specifies that the game uses a pyramid board with 3 rows (pyramid levels).

5.29.2 Member Function Documentation

5.29.2.1 create_player()

```
Player< char > * Pyramid_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a player of the specified type.

Factory method to create a player object.

Parameters

<i>name</i>	Name of the player.
<i>symbol</i>	Character symbol ('X' or 'O') assigned to the player.
<i>type</i>	The type of the player (Human or Computer).

Returns

Pointer to the newly created [Player<char>](#) instance.

Creates either a human or computer player with the specified parameters. Prints confirmation message of player creation.

Parameters

<i>name</i>	Reference to string containing the player's name
<i>symbol</i>	The symbol assigned to the player ('X' or 'O')
<i>type</i>	PlayerType enum indicating HUMAN or COMPUTER player

Returns

[Player<char>](#)* Pointer to newly created [Player](#) object

Warning

Caller is responsible for deallocating the returned [Player](#) object

Implements [UI< char >](#).

5.29.2.2 get_move()

```
Move< char > * Pyramid_UI::get_move (
    Player< char > * player) [override], [virtual]
```

Retrieves the next move from a player.

Gets a move from the player (human or computer)

Parameters

<i>player</i>	Pointer to the player whose move is being requested.
---------------	--

Returns

A pointer to a new [Move<char>](#) object representing the player's action.

For human players, prompts for position input via console and validates that the position is within the pyramid's playable area. For computer players, generates random valid positions within the pyramid.

Parameters

<i>player</i>	Pointer to the Player object making the move
---------------	--

Returns

[Move<char>](#)* Pointer to newly created [Move](#) object containing the player's move

Note

Human input is validated to ensure it's within the pyramid shape

Invalid moves trigger error messages and re-prompt the user

Computer moves are automatically validated before being returned

Warning

Caller is responsible for deallocating the returned [Move](#) object

See also

[isPrintable\(\)](#)

Implements [UI< char >](#).

5.29.2.3 setup_players()

`Player< char > ** Pyramid_UI::setup_players () [override], [virtual]`

Sets up both players for the game.

Prompts for player names and types (Human or Computer) for both `Player X` and `Player O`. Creates and returns an array of two player objects.

Returns

`Player<char>**` Array of two `Player` pointers [`Player X`, `Player O`]

Note

Players[0] is always `Player X`

Players[1] is always `Player O`

Warning

Caller is responsible for deallocating both the array and the player objects

See also

`create_player()`

`get_player_name()`

`get_player_type_choice()`

Reimplemented from `UI< char >`.

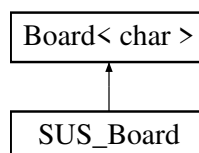
The documentation for this class was generated from the following files:

- include/PyramidTicTacToe_Classes.h
- src/ammen/PyramidTicTacToe_Classes.cpp

5.30 SUS_Board Class Reference

Represents the game board for the SUS game.

Inheritance diagram for `SUS_Board`:



Public Member Functions

- bool `update_board` (`Move< char > *move`)
Updates the board with a given move.
- bool `is_win` (`Player< char > *player`)
Determines if the given player has won.
- bool `is_lose` (`Player< char > *`)
Check if a player has lost.
- bool `is_draw` (`Player< char > *player`)
Determines if the game is a draw.
- bool `game_is_over` (`Player< char > *player`)
Checks if the game is over.
- int `calculate_score` (`char sym`)
Calculates the score for a given symbol.
- void `display_scores` ()

Public Member Functions inherited from Board< char >

- **Board** (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual **~Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from Board< char >

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.30.1 Detailed Description

Represents the game board for the SUS game.
Inherits from [Board](#) and manages moves, scoring, and game state.

5.30.2 Member Function Documentation

5.30.2.1 calculate_score()

```
int SUS_Board::calculate_score (
    char sym)
```

Calculates the score for a given symbol.

Checks all rows, columns, and diagonals for the "SUS" pattern. Only counts if the last move in the pattern belongs to the given symbol.

Parameters

<i>sym</i>	The symbol ('S' or 'U') to calculate score for.
------------	---

Returns

The score for the given symbol.

5.30.2.2 game_is_over()

```
bool SUS_Board::game_is_over (
    Player< char > * player) [virtual]
```

Checks if the game is over.

The game is over if either a player has won or the game is a draw.

Parameters

<i>player</i>	Pointer to the Player<char> object.
---------------	---

Returns

true if the game is over, false otherwise.

Implements [Board< char >](#).

5.30.2.3 is_draw()

```
bool SUS_Board::is_draw (
    Player< char > * player) [virtual]
```

Determines if the game is a draw.

A draw occurs when all cells are filled and no player has won.

Parameters

<i>player</i>	Pointer to the Player<char> object.
---------------	---

Returns

true if the game is a draw, false otherwise.

Implements [Board< char >](#).

5.30.2.4 is_lose()

```
bool SUS_Board::is_lose (
    Player< char > * ) [inline], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.30.2.5 is_win()

```
bool SUS_Board::is_win (
    Player< char > * player) [virtual]
```

Determines if the given player has won.

A player wins if their score is greater than the opponent's score after all moves are played.

Parameters

<i>player</i>	Pointer to the Player<char> object.
---------------	---

Returns

true if the player has won, false otherwise.

Implements [Board< char >](#).

5.30.2.6 update_board()

```
bool SUS_Board::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a given move.

Validates the move, applies it if valid, or undoes it if mark == 0.

Parameters

<i>move</i>	Pointer to the Move<char> object.
-------------	---

Returns

true if the move was successfully applied or undone, false otherwise.

Implements [Board<char>](#).

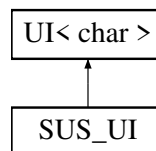
The documentation for this class was generated from the following files:

- include/SUS_Classes.h
- src/samir/SUS_Classes.cpp

5.31 SUS_UI Class Reference

Handles user interaction for the SUS game.

Inheritance diagram for SUS_UI:



Public Member Functions

- [Player<char>](#) * [setup_players](#) () override
Sets up the players for the SUS game.
- [Player<char>](#) * [create_player](#) (string &name, char symbol, PlayerType type)
Creates a player object.
- virtual [Move<char>](#) * [get_move](#) ([Player<char>](#) *player)
Gets the next move from a player.

Public Member Functions inherited from [UI<char>](#)

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI<char>](#)

- string [get_player_name](#) (string player_label)
Ask the user for the player's name.
- PlayerType [get_player_type_choice](#) (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI<char>](#)

- int [cell_width](#)
Width of each displayed board cell.

5.31.1 Detailed Description

Handles user interaction for the SUS game.
Provides methods to create players and get moves from them.

5.31.2 Member Function Documentation

5.31.2.1 create_player()

```
Player< char > * SUS_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a player object.

Parameters

<i>name</i>	The name of the player.
<i>symbol</i>	The symbol ('S' or 'U') assigned to the player.
<i>type</i>	The type of player (Human or Computer).

Returns

Pointer to the created [Player<char>](#) object.

Implements [UI< char >](#).

5.31.2.2 get_move()

```
Move< char > * SUS_UI::get_move (
    Player< char > * player) [virtual]
```

Gets the next move from a player.

If the player is human, prompts for input. If the player is computer, generates a random move.

Parameters

<i>player</i>	Pointer to the Player<char> object.
---------------	---

Returns

Pointer to the created [Move<char>](#) object.

Implements [UI< char >](#).

5.31.2.3 setup_players()

```
Player< char > ** SUS_UI::setup_players () [override], [virtual]
```

Sets up the players for the SUS game.

Prompts for player names and types (Human/Computer).

Returns

Pointer array of two [Player<char>](#) objects.

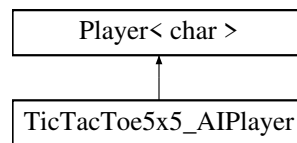
Reimplemented from [UI< char >](#).

The documentation for this class was generated from the following files:

- include/SUS_Classes.h
- src/samir/SUS_Classes.cpp

5.32 TicTacToe5x5_AIPlayer Class Reference

Inheritance diagram for TicTacToe5x5_AIPlayer:



Public Member Functions

- **TicTacToe5x5_AIPlayer** (string n, char s)
- pair< int, int > **get_best_move** ()

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- [Board](#)< char > * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** ([Board](#)< char > *b)
Assign the board pointer for the player.

Additional Inherited Members

Protected Attributes inherited from [Player< char >](#)

- string **name**
[Player](#) name.
- PlayerType **type**
[Player](#) type (e.g., HUMAN or COMPUTER)
- char **symbol**
Players symbol on board.
- [Board](#)< char > * **boardPtr**
Pointer to the game board.

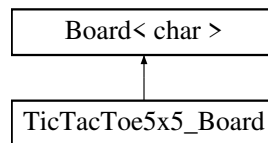
The documentation for this class was generated from the following files:

- include/TicTacToe5x5_Classes.h
- src/abdallah/TicTacToe5x5_Classes.cpp

5.33 TicTacToe5x5_Board Class Reference

Represents a 5x5 TicTacToe board with scoring logic for sequences of 3.

Inheritance diagram for TicTacToe5x5_Board:



Public Member Functions

- **TicTacToe5x5_Board** ()
Constructs a 5x5 TicTacToe board and initializes all cells as blank.
- bool **update_board** (Move< char > *move)
Updates the board with a new move if valid.
- bool **is_win** (Player< char > *player)
Checks if a given player has won the game.
- bool **is_lose** (Player< char > *player)
Check if a player has lost.
- bool **is_draw** (Player< char > *player)
Checks if the game ended in a draw.
- bool **game_is_over** (Player< char > *player)
Checks whether the game is over by win or draw.
- void **set_score** (Player< char > *player)
- void **get_score** (Player< char > *player)
- void **update_scores** ()
Updates player scores by recounting sequences of X and Y.
- void **desplay_scores** ()
Displays the current scores and announces the winner or draw.
- bool **end** ()
Determines if the game has reached the end (board full).

Public Member Functions inherited from Board< char >

- **Board** (int rows, int columns)
Construct a board with given dimensions.
- virtual ~**Board** ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > **get_board_matrix** () const
Return a copy of the current board as a 2D vector.
- int **get_rows** () const
Get number of rows.
- int **get_columns** () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from Board< char >

- int **rows**
Number of rows.
- int **columns**
Number of columns.
- vector< vector< char > > **board**
2D vector for the board
- int **n_moves**
Number of moves made.

5.33.1 Detailed Description

Represents a 5x5 TicTacToe board with scoring logic for sequences of 3.

5.33.2 Member Function Documentation

5.33.2.1 end()

```
bool TicTacToe5x5_Board::end ()
```

Determines if the game has reached the end (board full).

Returns

true if the game has ended.

false otherwise.

5.33.2.2 game_is_over()

```
bool TicTacToe5x5_Board::game_is_over (
    Player< char > * player) [virtual]
```

Checks whether the game is over by win or draw.

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if game is over.

false otherwise.

Implements [Board< char >](#).

5.33.2.3 is_draw()

```
bool TicTacToe5x5_Board::is_draw (
    Player< char > * player) [virtual]
```

Checks if the game ended in a draw.

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if draw.

false otherwise.

Implements [Board< char >](#).

5.33.2.4 is_lose()

```
bool TicTacToe5x5_Board::is_lose (
    Player< char > * ) [inline], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.33.2.5 is_win()

```
bool TicTacToe5x5_Board::is_win (
    Player< char > * player) [virtual]
```

Checks if a given player has won the game.

Parameters

<i>player</i>	Pointer to the current player.
---------------	--------------------------------

Returns

true if the player won.
false otherwise.

Implements [Board< char >](#).

5.33.2.6 update_board()

```
bool TicTacToe5x5_Board::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a new move if valid.

Parameters

<i>move</i>	Pointer to the Move object containing x, y, and symbol.
-------------	---

Returns

true if the move is valid and applied.
false if the move is invalid.

Implements [Board< char >](#).

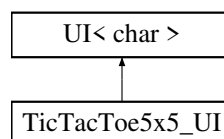
The documentation for this class was generated from the following files:

- include/TicTacToe5x5_Classes.h
- src/abdallah/TicTacToe5x5_Classes.cpp

5.34 TicTacToe5x5_UI Class Reference

Handles user interface for the 5x5 TicTacToe game.

Inheritance diagram for TicTacToe5x5_UI:

**Public Member Functions**

- **TicTacToe5x5_UI ()**
Constructs the [UI](#) and initializes the welcome message.
- **[Player](#)< char > * create_player** (string &name, char symbol, PlayerType type)
Creates a new player.
- virtual **[Move](#)< char > * get_move** (**[Player](#)**< char > *player)
Gets a move either from user input or computer AI.
- **[Player](#)< char > ** setup_players** ()
Set up players for the game.

Public Member Functions inherited from UI< char >

- **UI** (string message, int cell_display_width)
Construct the UI and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- void **display_board_matrix** (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from UI< char >

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from UI< char >

- int **cell_width**
Width of each displayed board cell.

5.34.1 Detailed Description

Handles user interface for the 5x5 TicTacToe game.

5.34.2 Member Function Documentation

5.34.2.1 create_player()

```
Player< char > * TicTacToe5x5_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a new player.

Parameters

<i>name</i>	Player name.
<i>symbol</i>	Player symbol (X or O).
<i>type</i>	HUMAN or COMPUTER.

Returns

Pointer to the created player.

Implements UI< char >.

5.34.2.2 get_move()

```
Move< char > * TicTacToe5x5_UI::get_move (
    Player< char > * player) [virtual]
```

Gets a move either from user input or computer AI.

Parameters

<i>player</i>	Pointer to current player.
---------------	----------------------------

Returns

[Move<char>](#)* Pointer to the created move.

Implements [UI< char >](#).

5.34.2.3 setup_players()

```
Player< char > ** TicTacToe5x5_UI::setup_players () [virtual]
```

Set up players for the game.

Default implementation of setting up two players.

Reimplemented from [UI< char >](#).

The documentation for this class was generated from the following files:

- include/TicTacToe5x5_Classes.h
- src/abdallah/TicTacToe5x5_Classes.cpp

5.35 Token Class Reference**Public Member Functions**

- void **move** (int y, int x)
- pair< int, int > **getPos** () const

The documentation for this class was generated from the following file:

- src/ammen/TicTacToe4x4_Classes.cpp

5.36 UI< T > Class Template Reference

Base class for handling user interface and input/output.

```
#include <BoardGame_Classes.h>
```

Public Member Functions

- **UI** (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void **display_message** (string message)
Display any message to the user.
- virtual [Move< T >](#) * **get_move** ([Player< T >](#) *)=0
Ask the user (or AI) to make a move.
- virtual [Player< T >](#) ** **setup_players** ()
Set up players for the game.
- virtual [Player< T >](#) * **create_player** (string &name, T symbol, PlayerType type)=0
Create a player object based on input name, symbol, and type.
- void **display_board_matrix** (const vector< vector< T > > &matrix) const
Display the current board matrix in formatted form.

Protected Member Functions

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes

- `int cell_width`
Width of each displayed board cell.

5.36.1 Detailed Description

`template<typename T>`
`class UI< T >`

Base class for handling user interface and input/output.

Template Parameters

<code>T</code>	The type of symbol used on the board.
----------------	---------------------------------------

5.36.2 Member Function Documentation**5.36.2.1 create_player()**

```
template<typename T>
virtual Player< T > * UI< T >::create_player (
    string & name,
    T symbol,
    PlayerType type) [pure virtual]
```

Create a player object based on input name, symbol, and type.

Implemented in [DimaondUI](#), [FourByFour_UI](#), [FourInARow_UI](#), [Infinity_TicTacToe_UI](#), [MemoryTicTacToeUI](#), [Mini_UI](#), [MisereTicTacToeUI](#), [NumericalTicTacToe_UI](#), [ObstaclesTicTacToeUI](#), [Pyramid_UI](#), [SUS_UI](#), [TicTacToe5x5_UI](#), [Ultimate_UI](#), [WordTicTacToe_UI](#), and [XO_UI](#).

5.36.2.2 get_move()

```
template<typename T>
virtual Move< T > * UI< T >::get_move (
    Player< T > * ) [pure virtual]
```

Ask the user (or AI) to make a move.

Implemented in [DimaondUI](#), [FourByFour_UI](#), [FourInARow_UI](#), [Infinity_TicTacToe_UI](#), [MemoryTicTacToeUI](#), [Mini_UI](#), [MisereTicTacToeUI](#), [NumericalTicTacToe_UI](#), [ObstaclesTicTacToeUI](#), [Pyramid_UI](#), [SUS_UI](#), [TicTacToe5x5_UI](#), [Ultimate_UI](#), [WordTicTacToe_UI](#), and [XO_UI](#).

5.36.2.3 setup_players()

```
template<typename T>
Player< T > ** UI< T >::setup_players () [virtual]
```

Set up players for the game.

Default implementation of setting up two players.

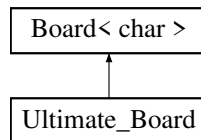
Reimplemented in [FourByFour_UI](#), [FourInARow_UI](#), [MisereTicTacToeUI](#), [NumericalTicTacToe_UI](#), [Pyramid_UI](#), [SUS_UI](#), [TicTacToe5x5_UI](#), and [WordTicTacToe_UI](#).

The documentation for this class was generated from the following file:

- `include/util/BoardGame_Classes.h`

5.37 Ultimate_Board Class Reference

Inheritance diagram for Ultimate_Board:



Public Member Functions

- [Ultimate_Board](#) ()
Ultimate Board.
- [~Ultimate_Board](#) ()
Destructor for [Ultimate_Board](#).
- bool [update_board](#) ([Move](#)< char > *move) override
Updates the ultimate board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player) override
Checks if the specified player has won the ultimate game.
- bool [is_draw](#) ([Player](#)< char > *player) override
Checks if the ultimate game has ended in a draw.
- bool [is_lose](#) ([Player](#)< char > *player) override
Check if a player has lost.
- bool [game_is_over](#) ([Player](#)< char > *player) override
Checks if the ultimate game has reached a terminal state.
- char [getMetaBoardCell](#) (int row, int col) const
Gets the value of a cell in the meta-board.
- [Mini_Board](#) * [getMiniBoard](#) (int row, int col) const
getters
- int [getActiveMiniRow](#) () const
Gets the row index of the currently active mini-board.
- int [getActiveMiniCol](#) () const
Gets the column index of the currently active mini-board.

Public Member Functions inherited from [Board](#)< char >

- [Board](#) (int [rows](#), int [columns](#))
Construct a board with given dimensions.
- virtual [~Board](#) ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > [get_board_matrix](#) () const
Return a copy of the current board as a 2D vector.
- int [get_rows](#) () const
Get number of rows.
- int [get_columns](#) () const
Get number of columns.

Friends

- class [Ultimate_UI](#)

Additional Inherited Members

Protected Attributes inherited from [Board< char >](#)

- **int rows**
Number of rows.
- **int columns**
Number of columns.
- **vector< vector< char > > board**
2D vector for the board
- **int n_moves**
Number of moves made.

5.37.1 Constructor & Destructor Documentation

5.37.1.1 Ultimate_Board()

`Ultimate_Board::Ultimate_Board ()`

Ultimate [Board](#).

Default constructor for [Ultimate_Board](#)

Initializes the 9x9 global board and creates 9 mini-boards in a 3x3 arrangement. Also initializes the meta-board that tracks mini-board winners.

Note

Sets activeMiniRow and activeMiniCol to -1, indicating no active board initially

Allocates memory for 9 [Mini_Board](#) objects

5.37.1.2 ~Ultimate_Board()

`Ultimate_Board::~~Ultimate_Board ()`

Destructor for [Ultimate_Board](#).

Deallocates memory for all 9 mini-boards.

5.37.2 Member Function Documentation

5.37.2.1 game_is_over()

`bool Ultimate_Board::game_is_over (`
 [Player](#)< char > * *player*) `[override], [virtual]`

Checks if the ultimate game has reached a terminal state.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the game is won or drawn

false if the game can continue

See also

[is_win\(\)](#)

[is_draw\(\)](#)

Implements [Board< char >](#).

5.37.2.2 getActiveMiniCol()

```
int Ultimate_Board::getActiveMiniCol () const
```

Gets the column index of the currently active mini-board.

Returns

int Active mini-board column (0-2), or -1 if no board is active

5.37.2.3 getActiveMiniRow()

```
int Ultimate_Board::getActiveMiniRow () const
```

Gets the row index of the currently active mini-board.

Returns

int Active mini-board row (0-2), or -1 if no board is active

5.37.2.4 getMetaBoardCell()

```
char Ultimate_Board::getMetaBoardCell (
    int row,
    int col) const
```

Gets the value of a cell in the meta-board.

Parameters

<i>row</i>	Meta-board row index (0-2)
<i>col</i>	Meta-board column index (0-2)

Returns

char The cell value ('X', 'O', 'D' for draw, or blank_symbol)

5.37.2.5 getMiniBoard()

```
Mini_Board * Ultimate_Board::getMiniBoard (
    int row,
    int col) const
```

getters

Gets a pointer to a specific mini-board

Parameters

<i>row</i>	Mini-board row index (0-2)
<i>col</i>	Mini-board column index (0-2)

Returns

Mini_Board* Pointer to the requested mini-board

5.37.2.6 is_draw()

```
bool Ultimate_Board::is_draw (
    Player< char > * player) [override], [virtual]
```

Checks if the ultimate game has ended in a draw.

A draw occurs when all mini-boards are decided (won or drawn) and neither player has won three mini-boards in a row.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if all mini-boards are decided with no winner
false if the game is still ongoing or someone has won

Implements [Board< char >](#).

5.37.2.7 is_lose()

```
bool Ultimate_Board::is_lose (
    Player< char > * ) [inline], [override], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.37.2.8 is_win()

```
bool Ultimate_Board::is_win (
    Player< char > * player) [override], [virtual]
```

Checks if the specified player has won the ultimate game.

Parameters

<i>player</i>	Pointer to the Player object to check for winning
---------------	---

Returns

true if the player has won three mini-boards in a row on the meta-board
false if no winning combination exists

See also

[checkMetaBoardWin\(\)](#)

Implements [Board< char >](#).

5.37.2.9 update_board()

```
bool Ultimate_Board::update_board (
    Move< char > * move) [override], [virtual]
```

Updates the ultimate board with a player's move.

Validates global coordinates, determines which mini-board to play in, enforces active board rules, updates the appropriate mini-board, and updates the meta-board if a mini-board becomes decided.

Parameters

<i>move</i>	Pointer to Move object containing global position (x,y) and player symbol
-------------	---

Returns

true if the move was valid and successfully placed
false if move is invalid (out of bounds, wrong mini-board, already decided, etc.)

Note

Global coordinates must be 0-8

Active board rules: players must continue in the board determined by opponent's last move

Prints error messages for invalid moves

See also

`isMiniboardPlayable()`

`updateMetaBoard()`

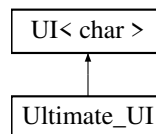
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/UltimateTicTacToe_Classes.h
- src/ammen/UltimateTicTacToe_Classes.cpp

5.38 Ultimate_UI Class Reference

Inheritance diagram for Ultimate_UI:

**Public Member Functions**

- [Ultimate_UI](#) ()
Default constructor for [Ultimate_UI](#).
- [Move](#)< char > * [get_move](#) ([Player](#)< char > *player) override
Gets a move from the player (human or computer)
- [Player](#)< char > * [create_player](#) (string &name, char symbol, PlayerType type) override
Factory method to create a player object.
- void [display_ultimate_board](#) ([Ultimate_Board](#) *board) const
Displays the complete ultimate board state.

Public Member Functions inherited from [UI](#)< char >

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- virtual [Player](#)< char > ** [setup_players](#) ()
Set up players for the game.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members**Protected Member Functions inherited from [UI](#)< char >**

- string [get_player_name](#) (string player_label)
Ask the user for the player's name.
- PlayerType [get_player_type_choice](#) (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- `int cell_width`
Width of each displayed board cell.

5.38.1 Constructor & Destructor Documentation

5.38.1.1 Ultimate_UI()

`Ultimate_UI::Ultimate_UI ()`

Default constructor for [Ultimate_UI](#).

Initializes the user interface with a welcome message and specifies that the game uses a 3x3 meta-board structure.

5.38.2 Member Function Documentation

5.38.2.1 create_player()

```
Player< char > * Ultimate_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [override], [virtual]
```

Factory method to create a player object.

Creates either a human or computer player with the specified parameters. Prints confirmation message of player creation.

Parameters

<i>name</i>	Reference to string containing the player's name
<i>symbol</i>	The symbol assigned to the player ('X' or 'O')
<i>type</i>	PlayerType enum indicating HUMAN or COMPUTER player

Returns

`Player<char>*` Pointer to newly created [Player](#) object

Warning

Caller is responsible for deallocating the returned [Player](#) object

Implements [UI< char >](#).

5.38.2.2 display_ultimate_board()

```
void Ultimate_UI::display_ultimate_board (
    Ultimate_Board * board) const
```

Displays the complete ultimate board state.

Shows the meta-board followed by all 9 mini-boards arranged in a 3x3 grid. Each mini-board is displayed side-by-side with its neighbors, and the active board is marked. Winners for each mini-board are shown below.

Parameters

<i>board</i>	Pointer to the Ultimate_Board to display
--------------	--

Note

This provides a comprehensive view of the entire game state

Active mini-board is marked with "*ACTIVE*"

Mini-board winners are displayed below each board

See also

`displayMetaBoard()`

5.38.2.3 `get_move()`

```
Move< char > * Ultimate_UI::get_move (
    Player< char > * player) [override], [virtual]
```

Gets a move from the player (human or computer)

For human players, displays which mini-board is active and prompts for global position input. For computer players, generates random valid positions within playable mini-boards.

Parameters

<code>player</code>	Pointer to the Player object making the move
---------------------	--

Returns

[Move](#)<char>* Pointer to newly created [Move](#) object containing the player's move

Note

Human input: global row and column (0-8)

Computer moves are randomly generated within playable mini-boards

Displays active board information with color coding (RED for required, GREEN for open)

Warning

Caller is responsible for deallocating the returned [Move](#) object

See also

`isMiniboardPlayable()`

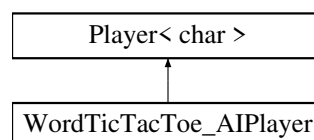
Implements [UI< char >](#).

The documentation for this class was generated from the following files:

- `include/UltimateTicTacToe_Classes.h`
- `src/ammen/UltimateTicTacToe_Classes.cpp`

5.39 WordTicTacToe_AIPlayer Class Reference

Inheritance diagram for WordTicTacToe_AIPlayer:



Public Member Functions

- **WordTicTacToe_AIPlayer** (string n, char s, [WordTicTacToe_board](#) *board)
- MoveEval **get_best_move** ()
- [Move](#)< char > * **make_ai_move** ()

Public Member Functions inherited from [Player< char >](#)

- **Player** (string n, char s, PlayerType t)
Construct a player with name, symbol, and type.
- string **get_name** () const
Get the player's name.
- PlayerType **get_type** () const
Get player type (e.g., 'H' or 'C').
- char **get_symbol** () const
Get the player's symbol.
- [Board< char >](#) * **get_board_ptr** () const
Get a pointer to the game board.
- void **set_board_ptr** ([Board< char >](#) *b)
Assign the board pointer for the player.

Additional Inherited Members**Protected Attributes inherited from [Player< char >](#)**

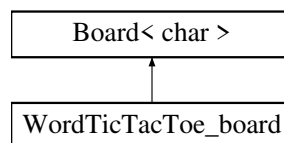
- string **name**
[Player](#) name.
- PlayerType **type**
[Player](#) type (e.g., HUMAN or COMPUTER)
- char **symbol**
Players symbol on board.
- [Board< char >](#) * **boardPtr**
Pointer to the game board.

The documentation for this class was generated from the following files:

- include/WordTicTacToe_Classes.h
- src/abdallah/[WordTicTacToe_Classes.cpp](#)

5.40 WordTicTacToe_board Class Reference

Inheritance diagram for WordTicTacToe_board:

**Public Member Functions**

- [WordTicTacToe_board](#) ()
Default constructor for [WordTicTacToe_board](#).
- [~WordTicTacToe_board](#) ()
Destructor for [WordTicTacToe_board](#).
- bool **update_board** ([Move< char >](#) *move)
Updates the board with a player's move.
- bool **is_win** ([Player< char >](#) *player)
Checks if the specified player has won the game.
- bool **is_lose** ([Player< char >](#) *player)

- *Check if a player has lost.*
- bool `is_draw` (Player< char > *player)
- *Checks if the game has ended in a draw.*
- bool `game_is_over` (Player< char > *player)
- *Checks if the game has reached a terminal state.*
- void `display_winner` (Player< char > *player)
- *Displays the winner announcement with the valid word.*
- void `display_draw` ()
- *Displays the draw game announcement.*
- const vector< string > & `get_dictionary` () const
- vector< vector< char > > `get_board_matrix` () const

Public Member Functions inherited from `Board< char >`

- **Board** (int rows, int columns)
- *Construct a board with given dimensions.*
- virtual ~**Board** ()
- *Virtual destructor. Frees allocated board memory.*
- vector< vector< char > > `get_board_matrix` () const
- *Return a copy of the current board as a 2D vector.*
- int `get_rows` () const
- *Get number of rows.*
- int `get_columns` () const
- *Get number of columns.*

Additional Inherited Members

Protected Attributes inherited from `Board< char >`

- int **rows**
- *Number of rows.*
- int **columns**
- *Number of columns.*
- vector< vector< char > > **board**
- *2D vector for the board*
- int **n_moves**
- *Number of moves made.*

5.40.1 Constructor & Destructor Documentation

5.40.1.1 `WordTicTacToe_board()`

`WordTicTacToe_board::WordTicTacToe_board ()`

Default constructor for `WordTicTacToe_board`.

Initializes a 3x3 board with all cells set to the blank symbol. Calls the base `Board` constructor with dimensions 3x3.

5.40.1.2 `~WordTicTacToe_board()`

`WordTicTacToe_board::~~WordTicTacToe_board ()`

Destructor for `WordTicTacToe_board`.

Cleans up resources allocated by the board object.

5.40.2 Member Function Documentation

5.40.2.1 display_winner()

```
void WordTicTacToe_board::display_winner (
    Player< char > * player)
```

Displays the winner announcement with the valid word.

Shows a formatted message with the winner's name and the word they formed. Uses green color for emphasis and waits for user input to continue.

Parameters

<code>player</code>	Pointer to the winning Player object
---------------------	--

Note

Uses ANSI color codes (GREEN, RESET) for terminal output

Pauses execution until user presses a key

5.40.2.2 display_draw()

```
void WordTicTacToe_board::display_draw ()
```

Displays the draw game announcement.

Shows a formatted message indicating the game ended without a winner. Uses yellow color for emphasis and waits for user input to continue.

Note

Uses ANSI color codes (YELLOW, RESET) for terminal output

Pauses execution until user presses Enter

5.40.2.3 game_is_over()

```
bool WordTicTacToe_board::game_is_over (
    Player< char > * player) [virtual]
```

Checks if the game has reached a terminal state.

Parameters

<code>player</code>	Pointer to the current Player object
---------------------	--

Returns

true if the game is won or drawn

false if the game can continue

See also

[is_win\(\)](#)

[is_draw\(\)](#)

Implements [Board< char >](#).

5.40.2.4 is_draw()

```
bool WordTicTacToe_board::is_draw (
    Player< char > * player) [virtual]
```

Checks if the game has ended in a draw.

A draw occurs when all 9 cells are filled and no player has won.

Parameters

<i>player</i>	Pointer to the current Player object
---------------	--

Returns

true if the game is a draw (9 moves made and no winner)
false if the game is not yet a draw

Note

Displays draw message when condition is met

See also

[display_draw\(\)](#)

Implements [Board< char >](#).

5.40.2.5 is_lose()

```
bool WordTicTacToe_board::is_lose (
    Player< char > * ) [inline], [virtual]
```

Check if a player has lost.

Implements [Board< char >](#).

5.40.2.6 is_win()

```
bool WordTicTacToe_board::is_win (
    Player< char > * player) [virtual]
```

Checks if the specified player has won the game.

Determines if a winning condition exists by checking for valid words on the board. Displays the winner if a valid word is found.

Parameters

<i>player</i>	Pointer to the Player object to check for winning
---------------	---

Returns

true if the player has formed a valid 3-letter word
false if no winning word has been formed

See also

[check_all_words\(\)](#)

[desplay_winner\(\)](#)

Implements [Board< char >](#).

5.40.2.7 update_board()

```
bool WordTicTacToe_board::update_board (
    Move< char > * move) [virtual]
```

Updates the board with a player's move.

Validates and places a letter on the board at the specified position. Performs bounds checking, occupancy checking, and letter validation.

Parameters

<code>move</code>	Pointer to Move object containing position (x,y) and letter symbol
-------------------	--

Returns

- true if the move was valid and successfully placed
- false if the move was invalid (out of bounds, occupied cell, or invalid letter)

Note

- Automatically converts letters to uppercase
- Increments the move counter on successful placement
- Prints error messages for invalid moves

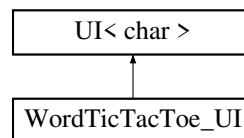
Implements [Board< char >](#).

The documentation for this class was generated from the following files:

- include/WordTicTacToe_Classes.h
- src/abdallah/WordTicTacToe_Classes.cpp

5.41 WordTicTacToe_UI Class Reference

Inheritance diagram for WordTicTacToe_UI:



Public Member Functions

- [WordTicTacToe_UI](#) ()
Default constructor for [WordTicTacToe_UI](#).
- [~WordTicTacToe_UI](#) ()
Destructor for [WordTicTacToe_UI](#).
- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Factory method to create a player object.
- virtual [Move< char > * get_move](#) ([Player< char > *player](#))
Gets a move from the player (human or computer)
- [Player< char > ** setup_players](#) ()
Set up players for the game.
- [WordTicTacToe_AIPlayer * create_ai_player](#) (string &name, char symbol, [WordTicTacToe_board *board](#))

Public Member Functions inherited from [UI< char >](#)

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members

Protected Member Functions inherited from [UI< char >](#)

- string **get_player_name** (string player_label)
Ask the user for the player's name.
- PlayerType **get_player_type_choice** (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI< char >](#)

- int **cell_width**
Width of each displayed board cell.

5.41.1 Constructor & Destructor Documentation

5.41.1.1 WordTicTacToe_UI()

`WordTicTacToe_UI::WordTicTacToe_UI ()`

Default constructor for [WordTicTacToe_UI](#).

Initializes the user interface with a welcome message and specifies that the game uses a 3x3 board.

5.41.1.2 ~WordTicTacToe_UI()

`WordTicTacToe_UI::~~WordTicTacToe_UI ()`

Destructor for [WordTicTacToe_UI](#).

Cleans up resources allocated by the [UI](#) object.

5.41.2 Member Function Documentation

5.41.2.1 create_player()

```
Player< char > * WordTicTacToe_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Factory method to create a player object.

Creates either a human or computer player with the specified parameters. Prints confirmation message of player creation.

Parameters

<i>name</i>	Reference to string containing the player's name
<i>symbol</i>	The symbol assigned to the player (unused in Word Tic-Tac-Toe)
<i>type</i>	PlayerType enum indicating HUMAN or COMPUTER player

Returns

[Player<char>](#)* Pointer to newly created [Player](#) object

Note

The symbol parameter is set to '?' as Word Tic-Tac-Toe uses letters, not fixed symbols

Warning

Caller is responsible for deallocating the returned [Player](#) object

Implements [UI< char >](#).

5.41.2.2 get_move()

```
Move< char > * WordTicTacToe_UI::get_move (
    Player< char > * player) [virtual]
```

Gets a move from the player (human or computer)

For human players, prompts for letter and position input via console. For computer players, generates random letter and position.

Parameters

<code>player</code>	Pointer to the Player object making the move
---------------------	--

Returns

[Move<char>](#)* Pointer to newly created [Move](#) object containing the player's move

Note

Human input: letter (A-Z) and position coordinates (0-2)

Computer move: randomly generated letter (A-Z) and position (0-2)

Warning

Caller is responsible for deallocating the returned [Move](#) object

Implements [UI< char >](#).

5.41.2.3 setup_players()

```
Player< char > ** WordTicTacToe_UI::setup_players () [virtual]
```

Set up players for the game.

Default implementation of setting up two players.

Reimplemented from [UI< char >](#).

The documentation for this class was generated from the following files:

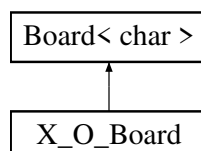
- include/WordTicTacToe_Classes.h
- src/abdallah/WordTicTacToe_Classes.cpp

5.42 X_O_Board Class Reference

Represents the Tic-Tac-Toe game board.

```
#include <PyramidTicTacToe_Classes.h>
```

Inheritance diagram for X_O_Board:



Public Member Functions

- [X_O_Board](#) ()
Default constructor that initializes a 3x3 X-O board.
- bool [update_board](#) ([Move](#)< char > *move)
Updates the board with a player's move.
- bool [is_win](#) ([Player](#)< char > *player)

- Checks if the given player has won the game.*
- bool `is_lose` (`Player`< char > *)
Checks if the given player has lost the game.
- bool `is_draw` (`Player`< char > *player)
Checks if the game has ended in a draw.
- bool `game_is_over` (`Player`< char > *player)
Determines if the game is over (win or draw).

Public Member Functions inherited from `Board`< char >

- `Board` (int `rows`, int `columns`)
Construct a board with given dimensions.
- virtual `~Board` ()
Virtual destructor. Frees allocated board memory.
- vector< vector< char > > `get_board_matrix` () const
Return a copy of the current board as a 2D vector.
- int `get_rows` () const
Get number of rows.
- int `get_columns` () const
Get number of columns.

Additional Inherited Members

Protected Attributes inherited from `Board`< char >

- int `rows`
Number of rows.
- int `columns`
Number of columns.
- vector< vector< char > > `board`
2D vector for the board
- int `n_moves`
Number of moves made.

5.42.1 Detailed Description

Represents the Tic-Tac-Toe game board.

This class inherits from the generic `Board`<char> class and implements the specific logic required for the Tic-Tac-Toe (X-O) game, including move updates, win/draw detection, and display functions.

See also

[Board](#)

5.42.2 Member Function Documentation

5.42.2.1 `game_is_over()`

```
bool X_O_Board::game_is_over (
    Player< char > * player) [virtual]
```

Determines if the game is over (win or draw).

Parameters

<code>player</code>	Pointer to the player to evaluate.
---------------------	------------------------------------

Returns

true if the game has ended, false otherwise.

Implements [Board< char >](#).

5.42.2.2 is_draw()

```
bool X_O_Board::is_draw (  
    Player< char > * player) [virtual]
```

Checks if the game has ended in a draw.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

true if all cells are filled and no player has won, false otherwise.

Implements [Board< char >](#).

5.42.2.3 is_lose()

```
bool X_O_Board::is_lose (  
    Player< char > * ) [inline], [virtual]
```

Checks if the given player has lost the game.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

Always returns false (not used in X-O logic).

Implements [Board< char >](#).

5.42.2.4 is_win()

```
bool X_O_Board::is_win (  
    Player< char > * player) [virtual]
```

Checks if the given player has won the game.

Parameters

<i>player</i>	Pointer to the player being checked.
---------------	--------------------------------------

Returns

true if the player has a winning line, false otherwise.

Implements [Board< char >](#).

5.42.2.5 update_board()

```
bool X_O_Board::update_board (  
    Move< char > * move) [virtual]
```

Updates the board with a player's move.

Parameters

<code>move</code>	Pointer to a Move<char> object containing move coordinates and symbol.
-------------------	--

Returns

true if the move is valid and successfully applied, false otherwise.

Implements [Board< char >](#).

The documentation for this class was generated from the following files:

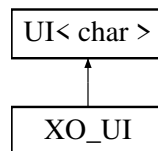
- include/XO_Classes.h
- src/group/XO_Classes.cpp

5.43 XO_UI Class Reference

User Interface class for the X-O (Tic-Tac-Toe) game.

```
#include <PyramidTicTacToe_Classes.h>
```

Inheritance diagram for XO_UI:

**Public Member Functions**

- [XO_UI](#) ()
Constructs an [XO_UI](#) object.
- [~XO_UI](#) ()
Destructor for [XO_UI](#).
- [Player< char > * create_player](#) (string &name, char symbol, PlayerType type)
Creates a player of the specified type.
- virtual [Move< char > * get_move](#) ([Player< char > *player](#))
Retrieves the next move from a player.

Public Member Functions inherited from [UI< char >](#)

- [UI](#) (string message, int cell_display_width)
Construct the [UI](#) and display a welcome message.
- void [display_message](#) (string message)
Display any message to the user.
- virtual [Player< char > ** setup_players](#) ()
Set up players for the game.
- void [display_board_matrix](#) (const vector< vector< char > > &matrix) const
Display the current board matrix in formatted form.

Additional Inherited Members**Protected Member Functions inherited from [UI< char >](#)**

- string [get_player_name](#) (string player_label)
Ask the user for the player's name.
- PlayerType [get_player_type_choice](#) (string player_label, const vector< string > &options)
Ask the user to choose the player type from a list.

Protected Attributes inherited from [UI<char>](#)

- `int cell_width`
Width of each displayed board cell.

5.43.1 Detailed Description

User Interface class for the X-O (Tic-Tac-Toe) game.

Inherits from the generic [UI<char>](#) base class and provides X-O specific functionality for player setup and move input.

See also

[UI](#)

Inherits from the generic [UI<char>](#) base class and provides X-O specific functionality for player setup and move input.

See also

[UI](#)

5.43.2 Constructor & Destructor Documentation

5.43.2.1 XO_UI()

```
XO_UI::XO_UI ()
```

Constructs an [XO_UI](#) object.

Initializes the base [UI<char>](#) class with the welcome message "FCAI X-O".

5.43.3 Member Function Documentation

5.43.3.1 create_player()

```
Player< char > * XO_UI::create_player (
    string & name,
    char symbol,
    PlayerType type) [virtual]
```

Creates a player of the specified type.

Parameters

<i>name</i>	Name of the player.
<i>symbol</i>	Character symbol ('X' or 'O') assigned to the player.
<i>type</i>	The type of the player (Human or Computer).

Returns

Pointer to the newly created [Player<char>](#) instance.

Implements [UI<char>](#).

5.43.3.2 get_move()

```
Move< char > * XO_UI::get_move (
    Player< char > * player) [virtual]
```

Retrieves the next move from a player.

Parameters

<i>player</i>	Pointer to the player whose move is being requested.
---------------	--

Returns

A pointer to a new [Move](#)<char> object representing the player's action.

Implements [UI](#)< char >.

The documentation for this class was generated from the following files:

- [include/XO_Classes.h](#)
- [src/group/XO_Classes.cpp](#)

Chapter 6

File Documentation

6.1 DiamondTicTacToe_Classes.h

```
00001 #pragma once
00002 #include <iostream>
00003 #include "util/BoardGame_Classes.h"
00004 #include <vector>
00005 #include <string>
00006
00007 class DimaondTicTacToe_Board : public Board<char> {
00008 public :
00009     DimaondTicTacToe_Board();
00010
00011     bool update_board(Move<char>* move) override;
00012     bool is_win(Player<char>* player) override;
00013     bool is_lose(Player<char>* player) override;
00014     bool is_draw(Player<char>* player) override;
00015     bool game_is_over(Player<char>* player) override;
00016     bool is_valid_cell(int x, int y);
00017 private :
00018     /*bool checkThree(char sym); */
00019     /*bool checkFive(char sym); */
00020
00021     bool checkThreeinrows(char sym);
00022     bool checkThreeincolumns(char sym);
00023     bool checkThreeindiagonals(char sym);
00024
00025     bool checkFourinrows(char sym);
00026     bool checkFourincolumns(char sym);
00027     bool checkFourindiagonals(char sym);
00028 };
00029
00030
00031 class DimaondPlayer : public Player<char>
00032 {
00033 public :
00034
00035     DimaondPlayer(string n, char s, PlayerType t) : Player(n, s, t) {}
00036 };
00037
00038 class DimaondUI : public UI<char> {
00039
00040 public :
00041     DimaondUI();
00042     Move<char>* get_move(Player<char>* player) override;
00043     Player<char>* create_player(string& name, char symbol, PlayerType type) override;
00044 };
```

6.2 FourInARow_Classes.h

```
00001 #ifndef FOURINAROW_CLASSES_H
00002 #define FOURINAROW_CLASSES_H
00003 #include "util/BoardGame_Classes.h"
00004 #include "util/colors.h"
00005 using namespace std;
00006
00007 class FourInARow_Board : public Board<char> {
00008 private:
00009     char blank_symbol = ' ';
00010
00011 public:
```

```

00012     FourInARow_Board();
00013     bool update_board(Move<char>* move);
00014     void reset_board();
00015     int calculate_score(char sym);
00016     bool is_win(Player<char>* player);
00017     bool is_lose(Player<char>*) override { return false; };
00018     char get_board_cell(int row, int col) const;
00019     bool is_draw(Player<char>* player);
00020     bool game_is_over(Player<char>* player);
00021     void print_board() const;
00022     void display_winner(Player<char>* player) const;
00023     void display_draw()const;
00024 };
00025
00026 class FourInARow_UI : public UI<char> {
00027 public:
00028     FourInARow_UI();
00029     ~FourInARow_UI() {};
00030     Player<char>* create_player(string& name, char symbol, PlayerType type);
00031     Move<char>* get_move(Player<char>* player);
00032     Player<char>* setup_players() override;
00033 };
00034
00035 void errorInvalidMove();
00036 void errorNoTokenHere(int y, int x);
00037 void debugPrint(const string& message);
00038
00039 #endif // FOURINAROW_CLASSES_H

```

6.3 InfinityTicTacToe_Classes.h

```

00001 #ifndef INFINITY_TICTACTOE_H
00002 #define INFINITY_TICTACTOE_H
00003 #include "util/BoardGame_Classes.h"
00004 #include <iostream>
00005 #include<queue>
00006 #include<utility>
00007 #include <util/colors.h>
00008 using namespace std;
00009
00010 class Infinity_TicTacToe_Board : public Board<char>{
00011
00012 public :
00013     Infinity_TicTacToe_Board();
00014     bool update_board(Move<char>* move) override;
00015     bool is_win(Player<char>* player) override;
00016     bool is_lose(Player<char>* player) override;
00017     bool is_draw(Player<char>* player) override;
00018     bool game_is_over(Player<char>* player) override;
00019     void display_winner(Player<char>* player);
00020     void display_draw();
00021 private:
00022     int counter = 0;
00023     bool checking(char sym);
00024     queue<pair<int, int>> moveQueue;
00025 };
00026
00027 class Infinity_TicTacToe_Player : public Player<char>
00028 {
00029 public:
00030     Infinity_TicTacToe_Player(string n, char s, PlayerType t): Player(n, s, t) {}
00031 };
00032
00033 class Infinity_TicTacToe_UI : public UI<char>
00034 {
00035 public:
00036     Infinity_TicTacToe_UI();
00037     Move<char>* get_move(Player<char>* player) override;
00038
00039     Player<char>* create_player(string& name, char symbol, PlayerType type) override;
00040 };
00041
00042
00043
00044 #endif INFINITY_TICTACTOE_H

```

6.4 maze.h

```

00001 //ifndef MAZE_SOLVER_H
00002 //define MAZE_SOLVER_H
00003 //

```

```

00004 // #include <string>
00005 // #include <vector>
00006 // #include <utility>
00007 //
00008 // typedef std::vector<std::vector<int>> Matrix;
00009 // typedef std::pair<int, int> Position;
00010 //
00012 // * @brief Main class for loading and solving mazes
00013 // *
00014 // * Cell values in matrix:
00015 // * 0 = empty space
00016 // * 1 = post (o)
00017 // * 2 = horizontal obstacle (-)
00018 // * 3 = vertical obstacle (|)
00019 // * 4 = goal (G)
00020 // * 5 = start (S)
00021 // * 8 = path marker (*)
00022 // * 9 = node (N)
00023 // */
00024 // class MazeSolver {
00025 // private:
00026 //     std::string mazePath;
00027 //     int totalRows;
00028 //     int totalCols;
00029 //     Position startPos;
00030 //     Position goalPos;
00031 //     bool mazeLoaded;
00032 //
00033 // public:
00034 //     /**
00035 //      * @brief Constructor
00036 //      */
00037 //     MazeSolver();
00038 //
00039 //     /**
00040 //      * @brief Load a maze from a file path
00041 //      * @param path Full path to the maze file (e.g., "/home/user/mazes/1.txt")
00042 //      * @return true if maze loaded successfully, false otherwise
00043 //      */
00044 //     bool loadMaze(const std::string& path);
00045 //
00046 //     /**
00047 //      * @brief Get the maze as a matrix of integers
00048 //      * @return Matrix representation of the maze
00049 //      */
00050 //     Matrix getMazeMatrix();
00051 //
00052 //     /**
00053 //      * @brief Get the maze with detected nodes marked
00054 //      * @return Matrix with nodes marked as 9
00055 //      */
00056 //     Matrix getMazeWithNodes();
00057 //
00058 //     /**
00059 //      * @brief Solve the maze using Dijkstra's algorithm
00060 //      * @return Matrix with the shortest path marked with 8,
00061 //      *         connecting line segments marked with 8 as well
00062 //      */
00063 //     Matrix solveMaze();
00064 //
00065 //     /**
00066 //      * @brief Check if a position is valid (in bounds and not an obstacle)
00067 //      * @param pos Position to check
00068 //      * @return true if valid move, false otherwise
00069 //      */
00070 //     bool isValidMove(const Position& pos);
00071 //
00072 //     /**
00073 //      * @brief Check if player has reached the goal
00074 //      * @param pos Current player position
00075 //      * @return true if at goal, false otherwise
00076 //      */
00077 //     bool isGoal(const Position& pos);
00078 //
00079 //     /**
00080 //      * @brief Get the start position
00081 //      * @return Position of 'S' in the maze
00082 //      */
00083 //     Position getStartPosition();
00084 //
00085 //     /**
00086 //      * @brief Get the goal position
00087 //      * @return Position of 'G' in the maze
00088 //      */
00089 //     Position getGoalPosition();
00090 //
00091 //     /**

```

```

00092 //      * @brief Get maze dimensions
00093 //      * @return pair of (rows, cols)
00094 //      */
00095 //      std::pair<int, int> getDimensions();
00096 //
00097 //      /**
00098 //      * @brief Print the maze matrix to console (for debugging)
00099 //      * @param matrix The matrix to print
00100 //      */
00101 //      static void printMatrix(const Matrix& matrix);
00102 //
00103 //      /**
00104 //      * @brief Get a human-readable character for a cell value
00105 //      * @param cellValue The integer value from the matrix
00106 //      * @return Character representation
00107 //      */
00108 //      static char cellToChar(int cellValue);
00109 //};
00110 //
00111 //endif // MAZE_SOLVER_H

```

6.5 MemoryTicTacToe_Classes.h

```

00001 #ifndef MemoryTicTacToe_CLASSES_H
00002 #define MemoryTicTacToe_CLASSES_H
00003
00004 #include "util/BoardGame_Classes.h"
00005 #include "util/colors.h"
00006 #include <vector>
00007 using namespace std;
00008
00009
00010 class MemoryTicTacToeBoard : public Board<char> {
00011 private:
00012     char blank_symbol = '.';
00013     char memory_symbol = '!';
00014     vector<vector<char>> memory_board;
00015 public:
00016     MemoryTicTacToeBoard();
00017     bool update_board(Move<char>* move);
00018     bool is_win(Player<char>* player);
00019     bool is_lose(Player<char>* player) { return false; }
00020     bool is_draw(Player<char>* player);
00021     bool game_is_over(Player<char>* player);
00022     void display_winner(Player<char>* player);
00023     void display_draw();
00024     void reveal_final_board();
00025 };
00026
00027
00028 class MemoryTicTacToeUI : public UI<char> {
00029 public:
00030     MemoryTicTacToeUI();
00031     ~MemoryTicTacToeUI() {}
00032     Player<char>* create_player(string& name, char symbol, PlayerType type);
00033     virtual Move<char>* get_move(Player<char>* player);
00034 };
00035
00036 #endif

```

6.6 MisereTicTacToe_Classes.h

```

00001 #ifndef MISERE_TICTACTOE_H
00002 #define MISERE_TICTACTOE_H
00003
00004 #include "util/BoardGame_Classes.h"
00005 #include <iostream>
00006 #include <utility>
00007
00008 using namespace std;
00009
00010 class MisereTicTacToeBoard : public Board<char> {
00011 public:
00012     MisereTicTacToeBoard();
00013
00014     bool update_board(Move<char>* move) override;
00015     bool is_win(Player<char>* player) override;
00016     bool is_lose(Player<char>* player) override;
00017     bool is_draw(Player<char>* player) override;
00018     bool game_is_over(Player<char>* player) override;
00019

```

```

00020 private:
00021     bool check_three_in_row(char sym);
00022 };
00023
00024 class MiserePlayer : public Player<char> {
00025 public:
00026     MiserePlayer(string n, char s, PlayerType t)
00027         : Player(n, s, t) {
00028     }
00029 };
00030
00031 // AI Player class
00032 class MisereAIPlayer : public Player<char> {
00033 private:
00034     int minimax(vector<vector<char>>& board, int depth, bool isMaximizing,
00035         char aiSymbol, char oppSymbol, int alpha, int beta);
00036
00037     static bool check_three_in_row_static(const vector<vector<char>>& b, char sym);
00038     static bool is_board_full(const vector<vector<char>>& b);
00039 public:
00040     MisereAIPlayer(string n, char s);
00041     pair<int, int> get_best_move();
00042 };
00043
00044 // UI for Misere Tic Tac Toe
00045 class MisereTicTacToeUI : public UI<char> {
00046 public:
00047     MisereTicTacToeUI();
00048     Move<char>* get_move(Player<char>* player) override;
00049     Player<char>* create_player(string& name, char symbol, PlayerType type) override;
00050     Player<char>** setup_players() override;
00051 };
00052
00053 #endif // MISERE_TICTACTOE_H

```

6.7 NumericalTicTacToe_Classes.h

```

00001 #pragma once
00002 #include "util/BoardGame_Classes.h"
00003 #include "util/colors.h"
00004 using namespace std;
00005
00006 class NumericalTicTacToe_Board : public Board<int> {
00007 private:
00008     int start_symbol = 0;
00009     bool used_numbers[10]; // 1 to 9 to track used digits
00010
00011 public:
00012     NumericalTicTacToe_Board();
00013
00014     bool update_board(Move<int>* move) override;
00015
00016     bool is_win(Player<int>* player) override;
00017
00018     bool is_lose(Player<int>* player) override { return false; }
00019
00020     bool is_draw(Player<int>* player) override;
00021
00022     bool game_is_over(Player<int>* player) override;
00023
00024     int calculate_score(int number);
00025
00026     bool is_valid_number(Player<int>* player, int number);
00027
00028     bool check_sum_15();
00029
00030     int get_cell(int r, int c) const;
00031
00032     bool is_used(int n) const;
00033
00034     void display_winner(Player<int>* player);
00035
00036     void display_draw();
00037
00038 };
00039
00040
00041
00042 class NumericalTicTacToe_UI : public UI<int> {
00043 private:
00044     NumericalTicTacToe_Board* board;
00045 public:

```

```

00047
00048     NumericalTicTacToe_UI(NumericalTicTacToe_Board* b);
00049
00050     ~NumericalTicTacToe_UI() {}
00051
00052     Player<int>* create_player(string& name, int symbol, PlayerType type) override;
00053
00054     Player<int>** setup_players() override;
00055
00056     Move<int>* get_move(Player<int>* player) override;
00057 };
00058

```

6.8 ObstaclesTicTacToe_Classes.h

```

00001
00002 #ifndef ObstaclesTicTacToe_CLASSES_H
00003 #define ObstaclesTicTacToe_CLASSES_H
00004
00005 #include "util/BoardGame_Classes.h"
00006 #include "util/colors.h"
00007 using namespace std;
00008
00009
00010 class ObstaclesTicTacToeBoard : public Board<char> {
00011 private:
00012     char blank_symbol = '.';
00013     char obstacle_symbol = '#';
00014 public:
00015     ObstaclesTicTacToeBoard();
00016     bool update_board(Move<char>* move);
00017     bool is_win(Player<char>* player);
00018     bool is_lose(Player<char>* player) { return false; };
00019     bool is_draw(Player<char>* player);
00020     bool game_is_over(Player<char>* player);
00021     void display_winner(Player<char>* player);
00022     void display_draw();
00023 };
00024
00025
00026 class ObstaclesTicTacToeUI : public UI<char> {
00027 public:
00028     ObstaclesTicTacToeUI();
00029     ~ObstaclesTicTacToeUI() {};
00030     Player<char>* create_player(string& name, char symbol, PlayerType type);
00031     virtual Move<char>* get_move(Player<char>* player);
00032 };
00033
00034 #endif

```

6.9 PyramidTicTacToe_Classes.h

```

00001 #ifndef PYRAMID_CLASSES_H
00002 #define PYRAMID_CLASSES_H
00003
00004 #include "util/BoardGame_Classes.h"
00005 #include "util/colors.h"
00006 using namespace std;
00007
00008 class Pyramid_Board : public Board<char> {
00009 private:
00010     char blank_symbol = ' ';
00011     static std::vector<std::pair<int, int>> printable;
00012 public:
00013     Pyramid_Board();
00014
00015     bool update_board(Move<char>* move);
00016
00017     bool is_win(Player<char>* player);
00018
00019     bool is_lose(Player<char>*) { return false; };
00020
00021     bool is_draw(Player<char>* player);
00022
00023     bool game_is_over(Player<char>* player);
00024
00025     friend bool isPrintable(int y, int x);
00026     friend void initPrintableLocations();
00027
00028     void display_winner(Player<char>* player);
00029

```

```

00068     void display_draw();
00069 };
00070
00071
00081 class Pyramid_UI : public UI<char> {
00082 public:
00088     Pyramid_UI();
00089
00093     ~Pyramid_UI() {};
00094
00102     Player<char>* create_player(string& name, char symbol, PlayerType type);
00103
00109     virtual Move<char>* get_move(Player<char>* player) override;
00110
00111
00112
00113     Player<char>** setup_players() override;
00114 };
00115
00116 #endif // XO_CLASSES_H

```

6.10 snake.h

```

00001

```

6.11 spaceship.h

```

00001

```

6.12 SUS_Classes.h

```

00001 #pragma once
00002 #include "util/BoardGame_Classes.h"
00003 #include "util/colors.h"
00004 #include <vector>
00005 using namespace std;
00006
00007
00008 class SUS_Board : public Board<char> {
00009 private:
00010     char start_symbol = '.';
00011     vector<vector<int>> move_order;
00012     bool displayed = false;
00013 public:
00014
00015     SUS_Board();
00016
00017     bool update_board(Move<char>* move);
00018
00019     bool is_win(Player<char>* player);
00020
00021     bool is_lose(Player<char>*) { return false; };
00022
00023     bool is_draw(Player<char>* player);
00024
00025     bool game_is_over(Player<char>* player);
00026     int calculate_score(char sym);
00027
00028     void display_scores();
00029
00030 };
00031
00032
00033
00034 class SUS_UI : public UI<char> {
00035 public:
00036
00037     SUS_UI();
00038
00039
00040     ~SUS_UI() {};
00041
00042     Player<char>** setup_players() override;
00043
00044     Player<char>* create_player(string& name, char symbol, PlayerType type);
00045
00046
00047     virtual Move<char>* get_move(Player<char>* player);

```

```
00048 };
```

6.13 TicTacToe4x4_Classes.h

```
00001
00010
00011 #ifndef FOURBYFOUR_CLASSES_H
00012 #define FOURBYFOUR_CLASSES_H
00013
00014 #include "util/BoardGame_Classes.h"
00015 #include "util/colors.h"
00016 using namespace std;
00017
00028 class FourByFour_Board : public Board<char> {
00029 private:
00030     char blank_symbol = ' ';
00031
00032 public:
00036     FourByFour_Board();
00037
00043     bool update_board(Move<char>* move);
00044
00050     bool is_win(Player<char>* player);
00051
00057     bool is_lose(Player<char>*) { return false; };
00058
00064     bool is_draw(Player<char>* player);
00065
00071     bool game_is_over(Player<char>* player);
00072
00073
00077     void putTokensAtPositions();
00078
00079     void display_winner(Player<char>* player);
00080     void display_draw();
00081 };
00082
00083
00093 class FourByFour_UI : public UI<char> {
00094 public:
00100     FourByFour_UI();
00101
00105     ~FourByFour_UI() {};
00106
00114     Player<char>* create_player(string& name, char symbol, PlayerType type);
00115
00121     virtual Move<char>* get_move(Player<char>* player);
00122
00123
00124     Player<char> **setup_players() override;
00125 };
00126
00127
00128
00129 #endif // XO_CLASSES_H
```

6.14 TicTacToe5x5_Classes.h

```
00001 #ifndef TicTacToe5x5_Classes_h
00002 #define TicTacToe5x5_Classes_h
00003
00004 #include "util/BoardGame_Classes.h"
00005 #include "util/colors.h"
00006 using namespace std;
00007
00008 class TicTacToe5x5_Board : public Board<char>
00009 {
00010 private:
00011     char blank_symbol = ' ';
00012     int player1_score = 0;
00013     int player2_score = 0;
00014     bool game_ended = false;
00015     int count_sequences(char symbol);
00016 public:
00017     TicTacToe5x5_Board();
00018     ~TicTacToe5x5_Board();
00019     bool update_board(Move<char>* move);
00020     bool is_win(Player<char>* player);
00021     bool is_lose(Player<char>* player) { return false; };
00022     bool is_draw(Player<char>* player);
00023     bool game_is_over(Player<char>* player);
```



```

00024     void set_score(Player<char>* player);
00025     void get_score(Player<char>* player);
00026     void update_scores();
00027     void display_scores();
00028     bool end();
00029 };
00030
00031 class TicTacToe5x5_AIPlayer : public Player<char> {
00032 private:
00033     int minimax(vector<vector<char>>& board, int depth, bool isMaximizing,
00034         char aiSymbol, char oppSymbol, int alpha, int beta, int maxDepth = 4);
00035
00036     static bool is_board_full(const vector<vector<char>>& b, int moves);
00037     static int count_sequences_in_board(const vector<vector<char>>& board, char symbol);
00038
00039 public:
00040     TicTacToe5x5_AIPlayer(string n, char s);
00041     pair<int, int> get_best_move();
00042 };
00043
00044
00045 class TicTacToe5x5_UI : public UI<char>
00046 {
00047 public:
00048     TicTacToe5x5_UI();
00049     ~TicTacToe5x5_UI();
00050     Player<char>* create_player(string& name, char symbol, PlayerType type);
00051     virtual Move<char>* get_move(Player<char>* player);
00052     Player<char>*> setup_players();
00053 };
00054 #endif

```

6.15 UltimateTicTacToe_Classes.h

```

00001
00009
00010 #ifndef ULTIMATE_CLASSES_H
00011 #define ULTIMATE_CLASSES_H
00012
00013 #include "util/BoardGame_Classes.h"
00014 #include "util/colors.h"
00015 using namespace std;
00016
00017
00018 class Mini_Board : public Board<char> {
00019 private:
00020     char winner = ' '; // 'X', 'O', or ' '
00021     char blank_symbol = ' ';
00022
00023 public:
00024     Mini_Board();
00025     bool update_board(Move<char>* move) override;
00026     bool is_win(Player<char>* player) override;
00027     bool is_draw(Player<char>* player) override;
00028     bool is_lose(Player<char>* player) override;
00029     bool game_is_over(Player<char>* player) override;
00030
00031     char get_winner() const;
00032     void set_winner(char w);
00033     bool is_decided() const;
00034 };
00035
00036
00037
00038
00039
00040 class Mini_UI : public UI<char> {
00041 public:
00042     Mini_UI();
00043     ~Mini_UI() {};
00044     Player<char>* create_player(string& name, char symbol, PlayerType type);
00045     virtual Move<char>* get_move(Player<char>* player);
00046 };
00047
00048
00049 // =====
00050 //
00051 //
00052 class Ultimate_Board : public Board<char> {
00053 private:
00054     friend class Ultimate_UI;
00055     char blank_symbol = ' ';
00056     Mini_Board* miniBoards[3][3];
00057     char metaBoard[3][3];

```

```

00068     int activeMiniRow, activeMiniCol; // -1 = any board
00069
00070     // Helper methods
00071     int getMiniRow(int globalX) const;
00072     int getMiniCol(int globalY) const;
00073     int getLocalX(int globalX) const;
00074     int getLocalY(int globalY) const;
00075
00076     void updateMetaBoard(int miniRow, int miniCol);
00077     bool isMiniboardPlayable(int miniRow, int miniCol) const;
00078     bool checkMetaBoardWin(char symbol) const;
00079
00080
00081 public:
00082     Ultimate_Board();
00083     ~Ultimate_Board(); // Delete miniBoards
00084
00085     bool update_board(Move<char>* move) override;
00086     bool is_win(Player<char>* player) override;
00087     bool is_draw(Player<char>* player) override;
00088     bool is_lose(Player<char>* player) override { return false; }
00089     bool game_is_over(Player<char>* player) override;
00090
00091
00092     char getMetaBoardCell(int row, int col) const;
00093     Mini_Board* getMiniBoard(int row, int col) const;
00094     int getActiveMiniRow() const;
00095     int getActiveMiniCol() const;
00096 };
00097
00107 class Ultimate_UI : public UI<char> {
00108 private:
00109     void displaySingleMiniBoard(Mini_Board* mini, int miniRow, int miniCol,
00110                                bool isActive) const;
00111     void displayMetaBoard(Ultimate_Board* board) const;
00112
00113 public:
00114     Ultimate_UI();
00115
00116     // Override pure virtual functions
00117     Move<char>* get_move(Player<char>* player) override;
00118     Player<char>* create_player(string& name, char symbol, PlayerType type) override;
00119
00120     // Custom display for Ultimate board
00121     void display_ultimate_board(Ultimate_Board* board) const;
00122 };
00123
00124
00125
00126
00127 #endif // XO_CLASSES_H
00128

```

6.16 BoardGame_Classes.h

```

00001 #ifndef _BOARDGAME_CLASSES_H
00002 #define _BOARDGAME_CLASSES_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <iostream>
00007 #include <iomanip>
00008 using namespace std;
00009
00011 // Forward declarations
00013
00014 template <typename T> class Player;
00015 template <typename T> class Move;
00016
00018 // Class declarations
00020
00024 enum class PlayerType {
00025     HUMAN,
00026     COMPUTER,
00027     AI,
00028     RANDOM
00029 };
00030
00039 template <typename T>
00040 class Board {
00041 protected:
00042     int rows;
00043     int columns;
00044     vector<vector<T>> board;

```

```

00045     int n_moves = 0;
00046
00047 public:
00051     Board(int rows, int columns)
00052         : rows(rows), columns(columns), board(rows, vector<T>(columns)) {}
00053
00057     virtual ~Board() {}
00058
00064     virtual bool update_board(Move<T>* move) = 0;
00065
00067     virtual bool is_win(Player<T>*) = 0;
00068
00070     virtual bool is_lose(Player<T>*) = 0;
00071
00073     virtual bool is_draw(Player<T>*) = 0;
00074
00076     virtual bool game_is_over(Player<T>*) = 0;
00077
00081     vector<vector<T>> get_board_matrix() const {
00082         return board;
00083     }
00084
00086     int get_rows() const { return rows; }
00087
00089     int get_columns() const { return columns; }
00090 };
00091
00092 //-----
00098 template <typename T>
00099 class Move {
00100     int x;
00101     int y;
00102     T symbol;
00103
00104 public:
00106     Move(int x, int y, T symbol) : x(x), y(y), symbol(symbol) {}
00107
00109     int get_x() const { return x; }
00110
00112     int get_y() const { return y; }
00113
00115     T get_symbol() const { return symbol; }
00116 };
00117
00118 //-----
00124 template <typename T>
00125 class Player {
00126 protected:
00127     string name;
00128     PlayerType type;
00129     T symbol;
00130     Board<T>* boardPtr;
00131
00132 public:
00136     Player(string n, T s, PlayerType t)
00137         : name(n), symbol(s), type(t), boardPtr(nullptr) {}
00138
00139     virtual ~Player() {}
00140
00142     string get_name() const { return name; }
00143
00145     PlayerType get_type() const { return type; }
00146
00148     T get_symbol() const { return symbol; }
00149
00151     Board<T>* get_board_ptr() const { return boardPtr; }
00152
00154     void set_board_ptr(Board<T>* b) { boardPtr = b; }
00155 };
00156
00157 //-----
00163 template <typename T>
00164 class UI {
00165 protected:
00166     int cell_width;
00167
00171     string get_player_name(string player_label) {
00172         string name;
00173         cout << "Enter " << player_label << " name: ";
00174         getline(cin >> ws, name);
00175         return name;
00176     }
00177
00181     PlayerType get_player_type_choice(string player_label, const vector<string>& options) {
00182         cout << "Choose " << player_label << " type:\n";
00183         for (size_t i = 0; i < options.size(); ++i)
00184             cout << i + 1 << ". " << options[i] << "\n";

```

```

00185         int choice;
00186         cin >> choice;
00187         return (choice==3)?PlayerType::AI:(choice == 2) ? PlayerType::COMPUTER : PlayerType::HUMAN;
00188     }
00189
00190 public:
00191     UI(string message, int cell_display_width)
00192         : cell_width(cell_display_width) {
00193         cout << message << endl;
00194     }
00195
00196     virtual ~UI() {}
00197
00198     void display_message(string message) { cout << message << "\n"; }
00199
00200     virtual Move<T>* get_move(Player<T>*) = 0;
00201
00202     virtual Player<T>** setup_players();
00203
00204     virtual Player<T>* create_player(string& name, T symbol, PlayerType type) = 0;
00205
00206     void display_board_matrix(const vector<vector<T>& matrix) const {
00207         if (matrix.empty() || matrix[0].empty()) return;
00208
00209         int rows = matrix.size();
00210         int cols = matrix[0].size();
00211
00212         cout << "\n";
00213         for (int j = 0; j < cols; ++j)
00214             cout << setw(cell_width + 1) << j;
00215         cout << "\n";
00216         for (int i = 0; i < rows; ++i) {
00217             cout << setw(2) << i << " |";
00218             for (int j = 0; j < cols; ++j)
00219                 cout << setw(cell_width) << matrix[i][j] << " |";
00220             cout << "\n";
00221         }
00222         cout << endl;
00223     }
00224 };
00225
00226 //-----
00227 template <typename T>
00228 class GameManager {
00229     Board<T>* boardPtr;
00230     Player<T>* players[2];
00231     UI<T>* ui;
00232
00233 public:
00234     GameManager(Board<T>* b, Player<T>* p[2], UI<T>* u)
00235         : boardPtr(b), ui(u) {
00236         players[0] = p[0];
00237         players[1] = p[1];
00238         players[0]->set_board_ptr(b);
00239         players[1]->set_board_ptr(b);
00240     }
00241
00242     void run() {
00243         ui->display_board_matrix(boardPtr->get_board_matrix());
00244         Player<T>* currentPlayer = players[0];
00245
00246         while (true) {
00247             for (int i : {0, 1}) {
00248                 currentPlayer = players[i];
00249                 Move<T>* move = ui->get_move(currentPlayer);
00250
00251                 while (!boardPtr->update_board(move))
00252                     move = ui->get_move(currentPlayer);
00253
00254                 ui->display_board_matrix(boardPtr->get_board_matrix());
00255
00256                 if (boardPtr->is_win(currentPlayer)) {
00257                     ui->display_message(currentPlayer->get_name() + " wins!");
00258                     return;
00259                 }
00260                 if (boardPtr->is_lose(currentPlayer)) {
00261                     ui->display_message(players[1 - i]->get_name() + " wins!");
00262                     return;
00263                 }
00264                 if (boardPtr->is_draw(currentPlayer)) {
00265                     ui->display_message("Draw!");
00266                     return;
00267                 }
00268             }
00269         }
00270     }
00271 };

```

```

00299 };
00300
00301 //-----
00305 template <typename T>
00306 Player<T>** UI<T>::setup_players() {
00307     Player<T>** players = new Player<T>*[2];
00308     vector<string> type_options = { "Human", "Computer" };
00309
00310     string nameX = get_player_name("Player X");
00311     PlayerType typeX = get_player_type_choice("Player X", type_options);
00312     players[0] = create_player(nameX, static_cast<T>('X'), typeX);
00313
00314     string nameO = get_player_name("Player O");
00315     PlayerType typeO = get_player_type_choice("Player O", type_options);
00316     players[1] = create_player(nameO, static_cast<T>('O'), typeO);
00317
00318     return players;
00319 }
00320
00321 #endif // _BOARDGAME_CLASSES_H

```

6.17 colors.h

```

00001 #ifndef COLORS_H
00002 #define COLORS_H
00003
00004 // =====
00005 // ANSI COLOR CODES Text & Background Styles
00006 // =====
00007
00008 // ----- Reset -----
00009 #define RESET          "\033[0m"
00010
00011 // ----- Regular Colors -----
00012 #define BLACK          "\033[30m"
00013 #define RED             "\033[31m"
00014 #define GREEN          "\033[32m"
00015 #define YELLOW         "\033[33m"
00016 #define BLUE           "\033[34m"
00017 #define MAGENTA        "\033[35m"
00018 #define CYAN           "\033[36m"
00019 #define WHITE          "\033[37m"
00020
00021 // ----- Bright (Bold) Colors -----
00022 #define BRIGHT_BLACK   "\033[90m"
00023 #define BRIGHT_RED     "\033[91m"
00024 #define BRIGHT_GREEN   "\033[92m"
00025 #define BRIGHT_YELLOW  "\033[93m"
00026 #define BRIGHT_BLUE    "\033[94m"
00027 #define BRIGHT_MAGENTA "\033[95m"
00028 #define BRIGHT_CYAN    "\033[96m"
00029 #define BRIGHT_WHITE   "\033[97m"
00030
00031 // ----- Background Colors -----
00032 #define BG_BLACK       "\033[40m"
00033 #define BG_RED         "\033[41m"
00034 #define BG_GREEN       "\033[42m"
00035 #define BG_YELLOW      "\033[43m"
00036 #define BG_BLUE        "\033[44m"
00037 #define BG_MAGENTA     "\033[45m"
00038 #define BG_CYAN        "\033[46m"
00039 #define BG_WHITE       "\033[47m"
00040
00041 // ----- Bright Background Colors -----
00042 #define BG_BRIGHT_BLACK "\033[100m"
00043 #define BG_BRIGHT_RED   "\033[101m"
00044 #define BG_BRIGHT_GREEN "\033[102m"
00045 #define BG_BRIGHT_YELLOW "\033[103m"
00046 #define BG_BRIGHT_BLUE  "\033[104m"
00047 #define BG_BRIGHT_MAGENTA "\033[105m"
00048 #define BG_BRIGHT_CYAN  "\033[106m"
00049 #define BG_BRIGHT_WHITE "\033[107m"
00050
00051 // ----- Text Styles -----
00052 #define BOLD            "\033[1m"
00053 #define DIM             "\033[2m"
00054 #define ITALIC          "\033[3m"
00055 #define UNDERLINE       "\033[4m"
00056 #define BLINK           "\033[5m"
00057 #define REVERSE         "\033[7m"
00058 #define HIDDEN          "\033[8m"
00059
00060
00061

```

```
00062 #endif
```

6.18 GameLaunchers.h

```
00001 #pragma once
00002
00003 namespace GameLaunchers {
00004     void launch_xo_game(); // we will remove this once done
00005
00006
00007     // our games
00008     void launch_sus_game();
00009     void launch_four_in_a_row();
00010     void launch_5x5_tic_tac_toe();
00011     void launch_word_tic_tac_toe();
00012     void launch_misere_tic_tac_toe();
00013     void launch_diamond_tic_tac_toe();
00014     void launch_4x4_tic_tac_toe();
00015     void launch_pyramid_tic_tac_toe();
00016
00017     // group games
00018     void launch_numerical_tic_tac_toe();
00019     void launch_obstacles_tic_tac_toe();
00020     void launch_infinity_tic_tac_toe();
00021
00022     // group bonus
00023     void launch_ultimate_tic_tac_toe();
00024     void launch_memory_tic_tac_toe();
00025
00026     // ammen: some extras
00027     //void launch_labyrinth();
00028     //void launch_spaceship();
00029     //void launch_snake();
00030 }
00031
```

6.19 Menu.h

```
00001 #ifndef MENU_H
00002 #define MENU_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include "colors.h"
00007 class Menu {
00008 private:
00009     std::vector<std::string> options;
00010     int currentSelection;
00011
00012     void clearScreen();
00013     int getKeyPress();
00014
00015 public:
00016     Menu();
00017     static bool noOptionChosen;
00018     void addOption(const std::string& option);
00019     int display(bool enhanced);
00020     bool kbhit();
00021     int displayMenu(bool enhancedMenu);
00022     int getOptionCount() const;
00023     ~Menu();
00024 };
00025
00026 #endif
```

6.20 util.h

```
00001 #ifndef UTIL_H
00002 #define UTIL_H
00003
00004
00005 #include <iostream>
00006 #include "colors.h"
00007
00008 namespace util {
00009     inline void errorInvalidMove() {
00010         std::cout << RED << "\n** ERROR: invalid move, please re-enter your move" << RESET;
00011     }
00012 }
```

```

00012
00013 inline void errorNoTokenHere(int y, int x) {
00014     std::cout << RED << "*** ERROR: no token at row " << y << " column " << x << RESET;
00015 }
00016
00017 inline void debugPrint(std::string message) {
00018     std::cout << GREEN << "** Debug: " << message << RESET << std::endl;
00019 }
00020 }
00021
00022 #endif
00023

```

6.21 WordTicTacToe_Classes.h

```

00001 #ifndef WordTicTacToe_Classes_h
00002 #define WordTicTacToe_Classes_h
00003
00004 #include "util/BoardGame_Classes.h"
00005 #include "util/colors.h"
00006 #include <vector>
00007 #include <string>
00008 using namespace std;
00009
00010 class WordTicTacToe_board : public Board<char>
00011 {
00012 private:
00013     char blank_symbol = '.';
00014
00015     string valid_word = "";
00016
00017     bool is_valid_word(const string& word);
00018     bool check_all_words();
00019     vector<string> dictionary = {
00020         "AAH", "AAL", "AAS", "ABA", "ABO", "ABS", "ABY", "ACE", "ACT", "ADD",
00021         "ADO", "ADS", "ADZ", "AFF", "AFT", "AGA", "AGE", "AGO", "AHA", "AID",
00022         "AIL", "AIM", "AIN", "AIR", "AIS", "AIT", "ALA", "ALB", "ALE", "ALL",
00023         "ALP", "ALS", "ALT", "AMA", "AMI", "AMP", "AMU", "ANA", "AND", "ANE",
00024         "ANI", "ANT", "ANY", "APE", "APT", "ARB", "ARC", "ARE", "ARF", "ARK",
00025         "ARM", "ARS", "ART", "ASH", "ASK", "ASP", "ASS", "ATE", "ATT", "AUK",
00026         "AVA", "AVE", "AVO", "AWA", "AWE", "AWL", "AWN", "AXE", "AYE", "AYS",
00027         "AZO", "BAA", "BAD", "BAG", "BAH", "BAL", "BAM", "BAN", "BAP", "BAR",
00028         "BAS", "BAT", "BAY", "BED", "BEE", "BEG", "BEL", "BEN", "BET", "BEY",
00029         "BIB", "BID", "BIG", "BIN", "BIO", "BIS", "BIT", "BIZ", "BOA", "BOB",
00030         "BOD", "BOG", "BOO", "BOP", "BOS", "BOT", "BOW", "BOX", "BOY", "BRA",
00031         "BRO", "BRR", "BUB", "BUD", "BUG", "BUM", "BUN", "BUR", "BUS", "BUT",
00032         "BUY", "BYE", "BYS", "CAB", "CAD", "CAM", "CAN", "CAP", "CAR", "CAT",
00033         "CAW", "CAY", "CEE", "CEL", "CEP", "CHI", "CIS", "COB", "COD", "COG",
00034         "COL", "CON", "COO", "COP", "COR", "COS", "COT", "COW", "COX", "COY",
00035         "COZ", "CRY", "CUB", "CUD", "CUE", "CUM", "CUP", "CUR", "CUT", "CWM",
00036         "DAB", "DAD", "DAG", "DAH", "DAK", "DAL", "DAM", "DAP", "DAW", "DAY",
00037         "DEB", "DEE", "DEL", "DEN", "DEV", "DEW", "DEX", "DEY", "DIB", "DID",
00038         "DIE", "DIG", "DIM", "DIN", "DIP", "DIS", "DIT", "DOC", "DOE", "DOG",
00039         "DOL", "DOM", "DON", "DOS", "DOT", "DOWN", "DRY", "DUB", "DUD",
00040         "DUE", "DUG", "DUI", "DUN", "DUO", "DUP", "DYE", "EAR", "EAT", "EAU",
00041         "EBB", "ECU", "EDH", "EEL", "EFF", "EFS", "EFT", "EGG", "EGO", "EKE",
00042         "ELD", "ELF", "ELK", "ELL", "ELM", "ELS", "EME", "EMF", "EMS", "EMU",
00043         "END", "ENG", "ENS", "EON", "ERA", "ERE", "ERG", "ERN", "ERR", "ERS",
00044         "ESS", "ETA", "ETH", "EVE", "EWE", "EYE", "FAD", "FAG", "FAN", "FAR",
00045         "FAS", "FAT", "FAX", "FAY", "FED", "FEE", "FEH", "FEM", "FEN", "FER",
00046         "FET", "FEU", "FEW", "FEY", "FEZ", "FIB", "FID", "FIE", "FIG", "FIL",
00047         "FIN", "FIR", "FIT", "FIX", "FIZ", "FLU", "FLY", "FOB", "FOE", "FOG",
00048         "FOH", "FON", "FOP", "FOR", "FOU", "FOX", "FOY", "FRO", "FRY", "FUB",
00049         "FUD", "FUG", "FUN", "FUR", "GAB", "GAD", "GAE", "GAG", "GAL", "GAM",
00050         "GAN", "GAP", "GAR", "GAS", "GAT", "GAY", "GED", "GEE", "GEL", "GEM",
00051         "GEN", "GET", "GEY", "GHI", "GIB", "GID", "GIE", "GIG", "GIN", "GIP",
00052         "GIT", "GNU", "GOA", "GOB", "GOD", "GOO", "GOR", "GOT", "GOX", "GOY",
00053         "GUL", "GUM", "GUN", "GUT", "GUV", "GUY", "GYM", "GYP", "HAD", "HAE",
00054         "HAG", "HAH", "HAJ", "HAM", "HAO", "HAP", "HAS", "HAT", "HAW", "HAY",
00055         "HEH", "HEM", "HEN", "HEP", "HER", "HES", "HET", "HEW", "HEX", "HEY",
00056         "HIC", "HID", "HIE", "HIM", "HIN", "HIP", "HIS", "HIT", "HMM", "HOB",
00057         "HOD", "HOE", "HOG", "HON", "HOP", "HOT", "HOW", "HOY", "HUB", "HUE",
00058         "HUG", "HUH", "HUM", "HUN", "HUP", "HUT", "HYP", "ICE", "ICH", "ICK",
00059         "ICY", "IDS", "IFF", "IFS", "ILK", "ILL", "IMP", "INK", "INN", "INS",
00060         "ION", "IRE", "IRK", "ISM", "ITS", "IVY", "JAB", "JAG", "JAM", "JAR",
00061         "JAW", "JAY", "JEE", "JET", "JEU", "JEW", "JIB", "JIG", "JIN", "JOB",
00062         "JOE", "JOG", "JOT", "JOW", "JOY", "JUG", "JUN", "JUS", "JUT", "KAB",
00063         "KAE", "KAF", "KAS", "KAT", "KAY", "KEA", "KEF", "KEG", "KEN", "KEP",
00064         "KEX", "KEY", "KHI", "KID", "KIF", "KIN", "KIP", "KIR", "KIT", "KOA",
00065         "KOB", "KOI", "KOP", "KOR", "KOS", "KUE", "LAB", "LAC", "LAD", "LAG",
00066         "LAM", "LAP", "LAR", "LAS", "LAT", "LAV", "LAW", "LAX", "LAY", "LEA",
00067         "LED", "LEE", "LEG", "LEI", "LEK", "LET", "LEU", "LEV", "LEX", "LEY",
00068         "LEZ", "LIB", "LID", "LIE", "LIN", "LIP", "LIS", "LIT", "LOB", "LOG",
00069         "LOO", "LOP", "LOT", "LOW", "LOX", "LUG", "LUM", "LUV", "LUX", "LYE",

```

```

00070 "MAC", "MAD", "MAE", "MAG", "MAN", "MAP", "MAR", "MAS", "MAT", "MAW",
00071 "MAX", "MAY", "MED", "MEL", "MEM", "MEN", "MET", "MEW", "MHO", "MIB",
00072 "MID", "MIG", "MIL", "MIM", "MIR", "MIS", "MIX", "MOA", "MOB", "MOC",
00073 "MOD", "MOG", "MOL", "MOM", "MON", "MOO", "MOP", "MOR", "MOS", "MOT",
00074 "MOW", "MUD", "MUG", "MUM", "MUN", "MUS", "MUT", "NAB", "NAE", "NAG",
00075 "NAH", "NAM", "NAN", "NAP", "NAW", "NAY", "NEB", "NEE", "NET", "NEW",
00076 "NIB", "NIL", "NIM", "NIP", "NIT", "NIX", "NOB", "NOD", "NOG", "NOH",
00077 "NOM", "NOO", "NOR", "NOS", "NOT", "NOW", "NTH", "NUB", "NUN", "NUS",
00078 "NUT", "OAF", "OAK", "OAR", "OAT", "OBE", "OBI", "OCA", "ODD", "ODE",
00079 "ODS", "OES", "OFF", "OFT", "OHM", "OHO", "OHS", "OIL", "OKA", "OKE",
00080 "OLD", "OLE", "OMS", "ONE", "ONS", "OOH", "OOT", "OPE", "OPS", "OPT",
00081 "ORA", "ORB", "ORC", "ORE", "ORS", "ORT", "OSE", "OUD", "OUR", "OUT",
00082 "OVA", "OWE", "OWL", "OWN", "OXO", "OXY", "PAC", "PAD", "PAH", "PAL",
00083 "PAM", "PAN", "PAP", "PAR", "PAS", "PAT", "PAW", "PAX", "PAY", "PEA",
00084 "PEC", "PED", "PEE", "PEG", "PEH", "PEN", "PEP", "PER", "PES", "PET",
00085 "PEW", "PHI", "PHT", "PIA", "PIC", "PIE", "PIG", "PIN", "PIP", "PIS",
00086 "PIT", "PIU", "PIX", "PLY", "POD", "POH", "POI", "POL", "POM", "POP",
00087 "POT", "POW", "POX", "PRO", "PRY", "PSI", "PUB", "PUD", "PUG", "PUL",
00088 "PUN", "PUP", "PUR", "PUS", "PUT", "PYA", "PYE", "PYX", "QAT", "QUA",
00089 "RAD", "RAG", "RAH", "RAJ", "RAM", "RAN", "RAP", "RAS", "RAT", "RAW",
00090 "RAX", "RAY", "REB", "REC", "RED", "REE", "REF", "REG", "REI", "REM",
00091 "REP", "RES", "RET", "REV", "REX", "RHO", "RIA", "RIB", "RID", "RIF",
00092 "RIG", "RIM", "RIN", "RIP", "ROB", "ROC", "ROD", "ROE", "ROM", "ROT",
00093 "ROW", "RUB", "RUE", "RUG", "RUM", "RUN", "RUT", "RYA", "RYE", "SAB",
00094 "SAC", "SAD", "SAE", "SAG", "SAL", "SAP", "SAT", "SAU", "SAW", "SAX",
00095 "SAY", "SEA", "SEC", "SEE", "SEG", "SEI", "SEL", "SEN", "SER", "SET",
00096 "SEW", "SEX", "SHA", "SHE", "SHH", "SHY", "SIB", "SIC", "SIM", "SIN",
00097 "SIP", "SIR", "SIS", "SIT", "SIX", "SKA", "SKI", "SKY", "SLY", "SOB",
00098 "SOD", "SOL", "SON", "SOP", "SOS", "SOT", "SOU", "SOW", "SOX", "SOY",
00099 "SPA", "SPY", "SRI", "STY", "SUB", "SUE", "SUM", "SUN", "SUP", "SUQ",
00100 "SYN", "TAB", "TAD", "TAE", "TAG", "TAJ", "TAM", "TAN", "TAO", "TAP",
00101 "TAR", "TAS", "TAT", "TAU", "TAV", "TAW", "TAX", "TEA", "TED", "TEE",
00102 "TEG", "TEL", "TEN", "TET", "TEW", "THE", "THO", "THY", "TIC", "TIE",
00103 "TIL", "TIN", "TIP", "TIS", "TIT", "TOD", "TOE", "TOG", "TOM", "TON",
00104 "TOO", "TOP", "TOR", "TOT", "TOW", "TOY", "TRY", "TSK", "TUB", "TUG",
00105 "TUI", "TUN", "TUP", "TUT", "TUX", "TWA", "TWO", "TYE", "UDO", "UGH",
00106 "UKE", "ULU", "UMM", "UMP", "UNS", "UPO", "UPS", "URB", "URD", "URN",
00107 "USE", "UTA", "UTS", "VAC", "VAN", "VAR", "VAS", "VAT", "VAU", "VAV",
00108 "VAW", "VEE", "VEG", "VET", "VEX", "VIA", "VIE", "VIG", "VIM", "VIS",
00109 "VOE", "VOW", "VOX", "VUG", "WAB", "WAD", "WAE", "WAG", "WAN", "WAP",
00110 "WAR", "WAS", "WAT", "WAW", "WAX", "WAY", "WEB", "WED", "WEE", "WEN",
00111 "WET", "WHA", "WHO", "WHY", "WIG", "WIN", "WIS", "WIT", "WIZ", "WOE",
00112 "WOG", "WOK", "WON", "WOO", "WOP", "WOS", "WOT", "WOW", "WRV", "WUD",
00113 "WYE", "WYN", "XIS", "YAH", "YAK", "YAM", "YAP", "YAR", "YAW", "YAY",
00114 "YEA", "YEH", "YEN", "YEP", "YES", "YET", "YEW", "YID", "YIN", "YIP",
00115 "YOB", "YOD", "YOK", "YOM", "YON", "YOU", "YOW", "YUK", "YUM", "YUP",
00116 "ZAG", "ZAP", "ZAX", "ZED", "ZEE", "ZEK", "ZIG", "ZIN", "ZIP", "ZIT",
00117 "ZOA", "ZOO"
00118 };
00119 public:
00120     WordTicTacToe_board();
00121     ~WordTicTacToe_board();
00122     bool update_board(Move<char>* move);
00123     bool is_win(Player<char>* player);
00124     bool is_lose(Player<char>* player) { return false; };
00125     bool is_draw(Player<char>* player);
00126     bool game_is_over(Player<char>* player);
00127     void display_winner(Player<char>* player);
00128     void display_draw();
00129
00130     const vector<string>& get_dictionary() const { return dictionary; }
00131     vector<vector<char>> get_board_matrix() const;
00132 };
00133
00134 class WordTicTacToe_AIPlayer : public Player<char> {
00135 private:
00136     WordTicTacToe_board* boardPtr;
00137
00138     struct MoveEval {
00139         int x;
00140         int y;
00141         char letter;
00142         int score;
00143     };
00144
00145     int minimax(vector<vector<char>>& board, int depth, bool isMaximizing,
00146         int alpha, int beta, int maxDepth = 3);
00147
00148     static bool is_valid_word_in_dict(const string& word, const vector<string>& dict);
00149     static int count_valid_words_in_board(const vector<vector<char>>& board, const vector<string>& dict);
00150     int evaluate_board_state(const vector<vector<char>>& board, const vector<string>& dict);
00151     int count_potential_words(const vector<vector<char>>& board, int x, int y,
00152         char letter, const vector<string>& dict);
00153
00154 public:
00155     WordTicTacToe_AIPlayer(string n, char s, WordTicTacToe_board* board);

```



```

00156     MoveEval get_best_move();
00157     Move<char>* make_ai_move();
00158 };
00159
00160 class WordTicTacToe_UI : public UI<char>
00161 {
00162 public:
00163     WordTicTacToe_UI();
00164     ~WordTicTacToe_UI();
00165     Player<char>* create_player(string& name, char symbol, PlayerType type);
00166     virtual Move<char>* get_move(Player<char>* player);
00167     Player<char>** setup_players();
00168
00169     WordTicTacToe_AIPlayer* create_ai_player(string& name, char symbol, WordTicTacToe_board* board);
00170 };
00171
00172 #endif

```

6.22 include/XO_Classes.h File Reference

Defines the X-O (Tic-Tac-Toe) specific classes that extend the generic board game framework.

```

#include "util/BoardGame_Classes.h"
#include "util/colors.h"

```

Classes

- class [X_O_Board](#)
Represents the Tic-Tac-Toe game board.
- class [XO_UI](#)
User Interface class for the X-O (Tic-Tac-Toe) game.

6.22.1 Detailed Description

Defines the X-O (Tic-Tac-Toe) specific classes that extend the generic board game framework.

This file provides:

- [X_O_Board](#): A specialized board class for the Tic-Tac-Toe game.
- [XO_UI](#): A user interface class tailored to X-O game setup and player interaction.

6.23 XO_Classes.h

[Go to the documentation of this file.](#)

```

00001
00002 #ifndef XO_CLASSES_H
00003 #define XO_CLASSES_H
00004
00005 #include "util/BoardGame_Classes.h"
00006 #include "util/colors.h"
00007 using namespace std;
00008
00009 class X_O_Board : public Board<char> {
00010 private:
00011     char blank_symbol = '.';
00012
00013 public:
00014     X_O_Board();
00015
00016     bool update_board(Move<char>* move);
00017
00018     bool is_win(Player<char>* player);
00019
00020     bool is_lose(Player<char>*) { return false; };
00021
00022     bool is_draw(Player<char>* player);
00023
00024     bool game_is_over(Player<char>* player);
00025 };
00026
00027
00028

```

```

00075 class XO_UI : public UI<char> {
00076 public:
00082     XO_UI();
00083
00087     ~XO_UI() {};
00088
00096     Player<char>* create_player(string& name, char symbol, PlayerType type);
00097
00103     virtual Move<char>* get_move(Player<char>* player);
00104 };
00105
00106 #endif // XO_CLASSES_H

```

6.24 src/abdallah/WordTicTacToe_Classes.cpp File Reference

Implementation of Word Tic-Tac-Toe game classes.

```

#include "WordTicTacToe_Classes.h"
#include <iostream>
#include <iomanip>
#include <cctype>
#include <fstream>
#include <algorithm>

```

6.24.1 Detailed Description

Implementation of Word Tic-Tac-Toe game classes.

Author

Abdallah Youssef

This file contains the implementation of the Word Tic-Tac-Toe game board, user interface, and game logic. Players place letters to form valid 3-letter words in rows, columns, or diagonals.

6.25 src/ammen/PyramidTicTacToe_Classes.cpp File Reference

Implementation of Pyramid Tic-Tac-Toe game classes.

```

#include <cstdint>
#include <iostream>
#include <iomanip>
#include <cctype>
#include <utility>
#include <vector>
#include "../include/PyramidTicTacToe_Classes.h"
#include "../include/util/util.h"

```

Functions

- bool [isPrintable](#) (int y, int x)
Checks if a position is within the pyramid's playable area.
- void [initPrintableLocations](#) ()
Initializes the list of valid playable positions on the pyramid board.

6.25.1 Detailed Description

Implementation of Pyramid Tic-Tac-Toe game classes.

Author

Ammen

This file contains the implementation of the Pyramid Tic-Tac-Toe game, which uses a pyramid-shaped board instead of the traditional 3x3 grid. The board has 9 playable positions arranged in a pyramid formation.

6.25.2 Function Documentation**6.25.2.1 initPrintableLocations()**

```
void initPrintableLocations ()
```

Initializes the list of valid playable positions on the pyramid board.

Populates the Pyramid_Board::printable vector with all 9 valid positions that form the pyramid shape:

- Row 2: position (2,2) - pyramid top
- Row 3: positions (3,1), (3,2), (3,3) - pyramid middle
- Row 4: positions (4,0), (4,1), (4,2), (4,3), (4,4) - pyramid base

Note

This function should be called once during board initialization

Modifies the static Pyramid_Board::printable vector

6.25.2.2 isPrintable()

```
bool isPrintable (
    int y,
    int x)
```

Checks if a position is within the pyramid's playable area.

Validates whether the given coordinates correspond to a valid position in the pyramid structure by checking against the list of printable positions.

Parameters

<i>y</i>	Row coordinate of the position to check
<i>x</i>	Column coordinate of the position to check

Returns

true if the position is playable (within pyramid boundaries)

false if the position is outside the pyramid or non-playable

Note

Uses the static Pyramid_Board::printable vector for validation

See also

[initPrintableLocations\(\)](#)

6.26 src/ammen/UltimateTicTacToe_Classes.cpp File Reference

Implementation of Ultimate Tic-Tac-Toe game classes.

```
#include <iostream>
#include <iomanip>
#include <cctype>
#include "../include/UltimateTicTacToe_Classes.h"
#include "../include/util/util.h"
```

6.26.1 Detailed Description

Implementation of Ultimate Tic-Tac-Toe game classes.

This file contains the implementation of Ultimate Tic-Tac-Toe, a complex variant where players compete on a 9x9 board divided into 9 mini-boards (3x3 each). Winning a mini-board claims that square on the meta-board. The objective is to win three mini-boards in a row on the meta-board.